

FAST – National University of Computer & Emerging Sciences
Department of Software Engineering
Course: SE2004 – Software Design & Architecture | Spring 2026

Software Requirements and Design Document
for
SCRAFMS
Smart Campus Resource Allocation and Fair-Use Management System

Prepared by	Mussa Raza (24I-3022), Ahmed Jamal (24I-3006), Muhammad Zain (24I-3126)
Organization	FAST-NUCES, Islamabad
Course Instructor	Ms. Laiba Imran
Date	April 2026
Version	V4.0 — Final Deliverable

Contents

Document Version History	3
1. Introduction	4
1.1 Purpose	4
1.2 Product Scope	4
1.3 Title	4
1.4 Objectives	4
1.5 Problem Statement	4
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Functions	6
2.3 List of Use Cases	6
2.4 Brief Use Cases	7
2.5 Extended Use Cases	12
2.6 Use Case Diagram	21
3. Other Non-Functional Requirements	22
3.1 Performance Requirements	22
3.2 Safety Requirements	22
3.3 Security Requirements	22
3.4 Software Quality Attributes	22
3.5 Business Rules	22
3.6 Operating Environment	23
3.7 User Interfaces	23
4. Domain Model	26
5. System Sequence Diagrams	28
6. Sequence Diagrams	49
7. Class Diagram	59
8. High-Level Architecture	60
8.1 Package Diagram	61
8.2 Deployment Diagram	61
8.3 Component Diagram	62
9. Work Division Table	63
9.1 Design Patterns Summary	63

Document Version History

Version	Date	Deliverable	Description
V1.0	Feb 2026	Deliverable 1	Initial Vision Document — project scope, stakeholders, major features
V2.0	Mar 2026	Deliverable 2	Brief & Extended Use Cases, Domain Model, Use Case Diagram
V3.0	Apr 2026	Deliverable 3	SSDs, Sequence Diagrams, Updated Domain Model, Class Diagram
V4.0	Apr 2026	Final Deliverable	Complete implementation — Web UI, Java Business Logic, SQL Server DB, Architecture Diagrams

1. Introduction

1.1 Purpose

This document specifies the software requirements and complete design artefacts for SCRAFMS (Smart Campus Resource Allocation and Fair-Use Management System). It covers the full design lifecycle from vision through use case analysis, domain modelling, system sequence diagrams, sequence diagrams, class diagram, and high-level architecture, culminating in a fully implemented Java web application.

The intended audience is the course instructor Ms. Laiba Imran, project evaluators, and the development team. The document serves as both a requirements specification and a design record demonstrating consistency between modelled artefacts and implemented code.

1.2 Product Scope

SCRAFMS is a university-level resource management platform that replaces manual and first-come-first-serve study room booking systems with an intelligent, fairness-based allocation engine. Three user roles are served: Students who request and use rooms, Resource Managers who register resources and manage violations, and System Administrators who configure allocation strategies and override decisions.

The system is implemented as a Java application with a web-based frontend running on localhost:8080, a pure Java business logic layer with no external frameworks, and a SQL Server Express database backend.

1.3 Title

SCRAFMS — Smart Campus Resource Allocation and Fair-Use Management System.

The system addresses the immediate problem of inequitable and inefficient allocation of shared academic study spaces on university campuses. It provides an automated, transparent, and configurable solution that tracks student behaviour, penalises misuse, and ensures equitable access through a dynamic fairness scoring mechanism.

1.4 Objectives

- Automate study room allocation using a configurable fairness-based scoring algorithm
- Enforce accountability through automatic no-show detection and penalty application
- Maintain a priority-based waiting queue with automatic slot promotion
- Provide three configurable allocation modes: Fair Use, First-Come-First-Serve, and Exam Mode
- Deliver role-specific web dashboards for Students, Resource Managers, and Administrators
- Maintain immutable audit and activity logs for all administrative actions
- Implement SHA-256 password hashing and session-based authentication for security

1.5 Problem Statement

The problem of inefficient and unfair allocation of shared campus study spaces affects university students and resource management staff across academic institutions. Traditional booking systems that rely on first-come-first-serve logic or manual approvals consistently produce inequitable outcomes: a small number of students monopolise high-demand rooms while others cannot access them, frequent no-shows waste reserved time slots that could be used by other students, and administrative staff bear the burden of resolving disputes and managing cancellations manually.

The impact of this problem includes resource underutilisation during peak study periods, student dissatisfaction with access fairness, and an inability to enforce accountability without significant manual effort. As student populations grow and academic space remains constrained, these inefficiencies become increasingly costly to institutional productivity and student outcomes.

A successful solution would automatically allocate resources using fairness-based logic that considers each student's usage history and reliability, dynamically handle booking conflicts through a priority queue, detect and penalise no-shows without manual intervention, and ensure equitable access across the student population without requiring constant administrative supervision. SCRAFMS addresses all of these requirements.

2. Overall Description

2.1 Product Perspective

SCRAFMS is a new, self-contained product developed for the SE2004 Software Design & Architecture course. It is not a replacement for an existing system but a proposed solution to a documented problem in university resource management.

The system comprises three integrated layers: a Web Presentation Layer (HTML/CSS/JS served via Java HttpServer on port 8080), a Business Logic Layer (pure Java, no external frameworks), and a Data Access Layer (JDBC + SQL Server Express). External interfaces include a SQL Server Express database instance and a web browser. There are no dependencies on external APIs or cloud services.

2.2 Product Functions

- Fair-Use Allocation Engine — computes a priority score for each booking request based on the student's fairness score and active allocation mode
- No-Show Detection — background daemon thread detects expired unattended bookings, applies penalties, and triggers queue promotion automatically on server startup
- Dynamic Waiting Queue — priority-ordered queue per time slot with automatic promotion when slots are released
- Role-Based Web Dashboards — separate HTML interfaces with server-side session authentication and role enforcement at the API layer
- Administrative Override — any active booking can be overridden by an administrator with a mandatory reason; all actions are immutably audit-logged
- Resource Registration — rooms and time slots registered through a validated admin/manager web interface
- Allocation Mode Switching — runtime mode switching (Fair Use, FCFS, Exam Mode) with immediate effect via GoF Strategy pattern
- Penalty and Restriction Management — automatic account restriction after repeated no-show violations

2.3 List of Use Cases

The table below lists all 15 use cases. The Implemented column indicates whether the use case is fully implemented as a primary deliverable (Yes), implemented indirectly as a supporting side-effect of another use case (Yes (indirect)), or not implemented (No). The Main UC column indicates whether the use case was a primary implementation goal for the assigned developer.

UC ID	Use Case Name	Primary Actor	Implemented	Main UC	Assigned To
UC-MR-01	Request Study Room	Student	Yes	Yes	Mussa Raza
UC-MR-02	View Study Rooms	Student	Yes (indirect)	No	Mussa Raza
UC-MR-03	Cancel Booking	Student	Yes	Yes	Mussa Raza

UC ID	Use Case Name	Primary Actor	Implemented	Main UC	Assigned To
UC-MR-04	View Booking History	Student	Yes (indirect)	No	Mussa Raza
UC-MR-05	Join Waiting List	Student	Yes	Yes	Mussa Raza
UC-06	View Fairness Score	Student	Yes (indirect)	No	Muhammad Zain
UC-07	Receive No-Show Penalty	System (Automated)	Yes	Yes	Muhammad Zain
UC-08	Override Booking Decision	System Administrator	Yes	Yes	Muhammad Zain
UC-09	Receive Booking Notification	Student, System	Yes (indirect)	No	Ahmed Jamal
UC-10	Manage Account Restriction	Resource Manager, System	Yes (indirect)	No	Muhammad Zain
UC-11	Register New Resource	System Administrator, Resource Manager	Yes	Yes	Muhammad Zain
UC-12	Verify Check-In	Student	Yes	Yes	Ahmed Jamal
UC-13	Configure Fairness Policy	Resource Manager	No	No	Ahmed Jamal
UC-14	Switch Allocation Mode	System Administrator	Yes	Yes	Ahmed Jamal
UC-15	Auto-Promote Queue Member	System (Automated)	Yes	Yes	Ahmed Jamal

2.4 Brief Use Cases

Note: Use cases marked as Yes (indirect) in the implementation column were realised as supporting functionality during implementation of their related primary use cases. They are fully documented here as required by the course specification.

Mussa Raza (241-3022) — Main UCs: UC-MR-01, UC-MR-03, UC-MR-05

Field	Detail
Use Case ID	UC-MR-01
Use Case Name	Request Study Room
Actors	Student
Type	Primary — Implemented (Main)
Description	A student selects a desired study room and time slot. The system evaluates eligibility (restriction status, usage limit), calculates a fairness priority score based on the student's history, and uses the active allocation strategy to either confirm the booking or place the student in the waiting queue.

Field	Detail
Use Case ID	UC-MR-02
Use Case Name	View Study Rooms
Actors	Student
Type	Primary — Implemented (Indirect)
Description	A student views a list of available study rooms and their time slots. The system retrieves availability from the database and displays rooms that are currently free. Implemented indirectly as part of the Request Study Room flow — the room selection panel on the student dashboard provides this functionality.

Field	Detail
Use Case ID	UC-MR-03
Use Case Name	Cancel Booking
Actors	Student
Type	Primary — Implemented (Main)
Description	A student cancels an existing confirmed or pending booking before the session begins. The system verifies ownership, removes the booking, updates room availability, and triggers automatic promotion of the next queued student via the Observer pattern.

Field	Detail
Use Case ID	UC-MR-04
Use Case Name	View Booking History
Actors	Student
Type	Primary — Implemented (Indirect)
Description	A student accesses a list of their past, current, and upcoming bookings. Implemented indirectly as part of the Cancel Booking flow — the My Bookings dashboard tab provides full booking history retrieval and display without requiring a separate implementation effort.

Field	Detail
Use Case ID	UC-MR-05
Use Case Name	Join Waiting List
Actors	Student

Field	Detail
Type	Primary — Implemented (Main)
Description	If a requested slot is fully booked, the student may join the waiting queue. The system checks for duplicate entries, calculates a priority score, assigns a queue position, and notifies the student. A student cannot join the queue for a slot they already hold a confirmed booking for.

Muhammad Zain (24I-3126) — Main UCs: UC-07, UC-08, UC-11

Field	Detail
Use Case ID	UC-06
Use Case Name	View Fairness Score
Actors	Student
Type	Primary — Implemented (Indirect)
Description	A student views their current fairness score on their profile dashboard. Implemented indirectly — the My Penalties dashboard page displays the student's current fairness score as a circular progress ring, along with violation history, without requiring a dedicated separate implementation.

Field	Detail
Use Case ID	UC-07
Use Case Name	Receive No-Show Penalty
Actors	Student, System (Automated)
Type	Primary — Implemented (Main)
Description	When a student fails to check in within the designated check-in window, the system automatically detects the absence, marks the booking as a no-show, reduces the fairness score by the configured penalty value, and sends a notification. If accumulated violations exceed the threshold, a temporary booking restriction is applied via the Chain of Responsibility pattern.

Field	Detail
Use Case ID	UC-08
Use Case Name	Override Booking Decision
Actors	System Administrator
Type	Primary — Implemented (Main)

Field	Detail
Description	In exceptional circumstances, an administrator manually overrides an existing booking. A mandatory reason (10-500 characters) must be provided. The system cancels the booking, notifies the displaced student with the reason, frees the slot, and records the action in the immutable audit log via the Command and Facade patterns.

Field	Detail
Use Case ID	UC-09
Use Case Name	Receive Booking Notification
Actors	Student, System (Automated)
Type	Secondary — Implemented (Indirect)
Description	The system automatically sends notifications at key booking lifecycle events: confirmation, queue promotion, cancellation, and penalty application. Implemented indirectly — NotificationService (Observer pattern) is triggered throughout the booking lifecycle by all primary use case flows without requiring a separate UC implementation.

Field	Detail
Use Case ID	UC-10
Use Case Name	Manage Account Restriction
Actors	Resource Manager, System (Automated)
Type	Primary — Implemented (Indirect)
Description	When a student accumulates no-show violations beyond the configured threshold, the system automatically restricts their account. Implemented indirectly via the Chain of Responsibility penalty handling chain in UC-07 — RestrictionHandler applies and records the restriction automatically when the violation threshold is reached.

Field	Detail
Use Case ID	UC-11
Use Case Name	Register New Resource
Actors	System Administrator/Resource Manager
Type	Primary — Implemented (Main)
Description	A system administrator registers a new study room by providing room ID, name, building, location, capacity, and available time slots. The system validates all fields, checks for duplicate IDs, persists the record using the Command pattern, and immediately makes the room available for student booking.

Ahmed Jamal (24I-3006) — Main UCs: UC-12, UC-14, UC-15

Field	Detail
Use Case ID	UC-12
Use Case Name	Verify Check-In
Actors	Student
Type	Primary — Implemented (Main)
Description	A student confirms physical presence by checking in within the designated time window (configurable, default 15 minutes around booking start time). The system validates ownership and window validity, updates booking status to Active, records the check-in timestamp, and logs the event via the Template Method pattern.

Field	Detail
Use Case ID	UC-13
Use Case Name	Configure Fairness Policy
Actors	Resource Manager
Type	Primary — Not Implemented
Description	A resource manager adjusts fairness formula weights, no-show penalty values, and booking restriction thresholds through the policy configuration interface. This use case is fully designed and documented but was not implemented within the project timeline. The fairness policy table exists in the database and is read by the allocation engine, but no UI or API endpoint for modifying it was developed.

Field	Detail
Use Case ID	UC-14
Use Case Name	Switch Allocation Mode
Actors	System Administrator
Type	Primary — Implemented (Main)
Description	A system administrator switches the active allocation strategy between Fair-Use Mode, First-Come-First-Serve Mode, and Exam Mode. The system validates the request, persists the change, logs it in the audit trail, and injects the new strategy into the allocation service at runtime via the GoF Strategy pattern.

Field	Detail
Use Case ID	UC-15
Use Case Name	Auto-Promote Queue Member

Field	Detail
Actors	System (Automated)
Type	Primary — Implemented (Main)
Description	When a confirmed booking is cancelled or marked as a no-show, the system automatically identifies the highest-priority eligible student in the waiting queue for that slot, creates a confirmed booking, removes the queue entry, and sends a notification to the promoted student.

2.5 Extended Use Cases

The following section presents the extended use case specifications for all 15 use cases. Each entry details the primary actor, preconditions, postconditions, main scenario, extension scenarios, and the GRASP and GoF patterns applied during implementation.

UC-MR-01: Request Study Room

Field	Detail
Use Case ID	UC-MR-01
Use Case Name	Request Study Room
Primary Actor	Student
Preconditions	Student has a valid account, is not restricted, and has not exceeded the weekly booking limit. System is online and at least one room has available time slots.
Postconditions	Booking is confirmed or student is placed in the waiting queue. Decision is persisted and student is notified.
Main Scenario	1. Student selects room and time slot from the web dashboard. 2. System checks eligibility (restriction status, weekly limit). 3. System calculates fairness priority score via FairnessScoreCalculator. 4. Active allocation strategy (Fair Use / FCFS / Exam Mode) determines outcome. 5. If slot available: booking confirmed, slot marked unavailable, BookingRepository saves record. 6. If slot unavailable: student added to waiting queue with priority score. 7. Result communicated to student as CONFIRMED, QUEUED, RESTRICTED, or EXAM_BLOCKED.
Extensions	1a. Student is restricted — request rejected with RESTRICTED status. 1b. Slot already taken — student queued instead. 1c. Exam Mode active — request rejected with EXAM_BLOCKED status. No booking record inserted.
Patterns Applied	GRASP: Controller (BookingController), Pure Fabrication (EligibilityService, FairnessScoreCalculator, AllocationService). GoF: Strategy (FairUseStrategy, FCFSStrategy, ExamModeStrategy injected at runtime).

UC-MR-02: View Study Rooms

Field	Detail
Use Case ID	UC-MR-02
Use Case Name	View Study Rooms
Primary Actor	Student
Preconditions	Student is logged in. Study rooms are registered in the database.
Postconditions	List of available rooms and time slots displayed to student.
Main Scenario	1. Student opens the Request Room page on the student dashboard. 2. System retrieves room list from RoomRepository. 3. Student selects a room; system fetches available time slots from TimeSlotRepository. 4. Available slots displayed in the right panel. 5. Activity event logged.
Extensions	1a. No rooms available — empty state message displayed. 1b. No available slots for selected room — right panel shows no slots.
Patterns Applied	GRASP: Controller (BookingController), Information Expert (RoomRepository, TimeSlotRepository). Implemented as a supporting flow of UC-MR-01.

UC-MR-03: Cancel Booking

Field	Detail
Use Case ID	UC-MR-03
Use Case Name	Cancel Booking
Primary Actor	Student
Preconditions	Student is logged in and has an existing booking that has not yet started.
Postconditions	Booking removed, slot freed, queue promotion triggered, student notified.
Main Scenario	1. Student opens My Bookings on the student dashboard. 2. System retrieves bookings via BookingRepository. 3. Student clicks Cancel on a CONFIRMED or PENDING booking. 4. BookingValidator verifies ownership and status. 5. Booking deleted, slot freed via RoomRepository. 6. ActivityLog entry created. 7. WaitingQueueService.promoteNext(slotId) triggered. 8. NotificationService notifies student of cancellation confirmation.
Extensions	1a. Booking already ACTIVE — cancellation blocked, error shown. 1b. Booking not found — error shown.
Patterns Applied	GRASP: Controller (CancellationController), Pure Fabrication (BookingValidator), Information Expert (BookingRepository, RoomRepository). GoF: Observer (NotificationService observes CANCELLATION_CONFIRMED event).

UC-MR-04: View Booking History

Field	Detail
Use Case ID	UC-MR-04
Use Case Name	View Booking History
Primary Actor	Student
Preconditions	Student is logged in. Booking records exist in the database.
Postconditions	Student can view all past, current, and upcoming bookings in chronological order.
Main Scenario	1. Student opens My Bookings tab on the student dashboard. 2. System calls GET /api/bookings/history. 3. BookingController.getStudentBookings(studentId) retrieves all bookings. 4. Bookings displayed as a table with status badges. 5. Student can identify bookings eligible for cancellation or check-in.
Extensions	1a. No bookings found — empty state message shown.
Patterns Applied	GRASP: Controller (BookingController), Information Expert (BookingRepository). Implemented as a supporting flow of UC-MR-03 via the My Bookings dashboard tab.

UC-MR-05: Join Waiting List

Field	Detail
Use Case ID	UC-MR-05
Use Case Name	Join Waiting List
Primary Actor	Student
Preconditions	Student is logged in. Selected slot is fully booked. Student does not already hold a booking or queue entry for this slot.
Postconditions	Student added to waiting queue with priority score and queue position assigned.
Main Scenario	1. Student opens Waiting List tab and selects a room and unavailable slot. 2. System confirms slot is unavailable via RoomRepository. 3. WaitingQueueRepository checks for duplicate queue entry. 4. System checks student does not already have a confirmed booking for this slot. 5. FairnessScoreCalculator computes priority score. 6. Student added to queue with position assigned. 7. NotificationService sends queue registration confirmation.
Extensions	1a. Student already in queue for this slot — duplicate prevented, current position shown. 1b. Student already has a confirmed booking for this slot — queue join blocked with message.
Patterns Applied	GRASP: Controller (WaitingListController), Information Expert (WaitingQueueRepository), Pure Fabrication (FairnessScoreCalculator). GoF: Observer (NotificationService on QUEUE_REGISTERED event).

UC-06: View Fairness Score

Field	Detail
Use Case ID	UC-06
Use Case Name	View Fairness Score
Primary Actor	Student
Preconditions	Student is logged in. Student record exists in the Students table.
Postconditions	Student's current fairness score and violation history are displayed.
Main Scenario	1. Student opens My Penalties tab on the student dashboard. 2. System calls GET /api/auth/me to retrieve current fairnessScore and noShowCount. 3. System calls GET /api/penalties to retrieve violation log entries. 4. Fairness score displayed as a circular progress ring (green >70, yellow 40-70, red <40). 5. Account restriction status displayed as a status card. 6. Violation history displayed as a table.
Extensions	1a. No violations yet — violations table shows empty state message.
Patterns Applied	GRASP: Information Expert (StudentRepository, ViolationLogRepository). Implemented indirectly via the My Penalties dashboard tab as part of the no-show penalty system.

UC-07: Receive No-Show Penalty

Field	Detail
Use Case ID	UC-07
Use Case Name	Receive No-Show Penalty
Primary Actor	System (Automated)
Preconditions	A confirmed booking exists. The check-in window has fully elapsed. The student has not checked in. No manager override exists for this booking.
Postconditions	Booking marked NO_SHOW. Fairness score reduced. noShowCount incremented. Violation logged. If threshold exceeded, account restricted. Slot freed and queue promotion triggered.
Main Scenario	1. Background daemon thread fires on server startup. 2. NoShowController.processExpiredCheckIns() invoked. 3. NoShowService retrieves expired unchecked bookings from BookingRepository. 4. For each booking: NoShowService validates no manager override exists. 5. PenaltyService.applyPenalty(studentId, penaltyValue) called. 6. StudentRepository.updatePenalty() deducts fairness score, increments noShowCount. 7. Penalty request passed through handler chain: ViolationThresholdHandler → RestrictionHandler → NotificationHandler. 8. ViolationLogRepository saves violation record. 9. If noShowCount reaches threshold: StudentRepository.applyRestriction() sets isRestricted=true. 10. Booking marked NO_SHOW. Slot freed. WaitingQueueService.promoteNext() called.
Extensions	1a. Manager override exists — no-show processing skipped for this booking. 1b. Violation count reaches threshold — account restriction applied automatically. 1c. Penalty value is zero — violation logged but no score deduction.

Field	Detail
Patterns Applied	GRASP: Controller (NoShowController), Pure Fabrication (PenaltyService, NoShowService), Information Expert (BookingRepository, StudentRepository). GoF: Chain of Responsibility (PenaltyHandler → ViolationThresholdHandler → RestrictionHandler → NotificationHandler).

UC-08: Override Booking Decision

Field	Detail
Use Case ID	UC-08
Use Case Name	Override Booking Decision
Primary Actor	System Administrator
Preconditions	Administrator is logged in. Target booking exists and is in CONFIRMED, PENDING, or ACTIVE status. Valid override reason provided (10-500 characters).
Postconditions	Booking marked OVERRIDDEN. Slot freed. Displaced student notified. Audit log entry created. Queue promotion triggered.
Main Scenario	1. Admin opens Override Bookings page on admin dashboard. 2. System retrieves all active bookings via GET /api/override/bookings. 3. Admin clicks Override on a booking, types a reason. 4. OverrideValidator validates reason length (10-500 chars). 5. OverrideController.initiateOverride() called. 6. OverrideBookingCommand.execute() marks booking as OVERRIDDEN. 7. OverrideCompletionFacade.completeOverride() orchestrates: notify displaced student, update score, promote queue, log audit. 8. AuditLogRepository persists immutable audit entry.
Extensions	1a. Reason under 10 characters — Confirm button disabled, character counter shown in red. 1b. Booking already OVERRIDDEN — action blocked.
Patterns Applied	GRASP: Controller (OverrideController), Information Expert (OverrideValidator, BookingRepository), Low Coupling (OverrideCompletionFacade). GoF: Command (OverrideBookingCommand), Facade (OverrideCompletionFacade).

UC-09: Receive Booking Notification

Field	Detail
Use Case ID	UC-09
Use Case Name	Receive Booking Notification
Primary Actor	Student, System (Automated)
Preconditions	Student has an active account. A booking lifecycle event has occurred.
Postconditions	Student receives an accurate in-app notification. Notification record persisted.
Main Scenario	1. Booking lifecycle event fires (CONFIRMED, CANCELLATION_CONFIRMED, QUEUE_REGISTERED, NO_SHOW_PENALTY, QUEUE_PROMOTED). 2.

Field	Detail
	NotificationService.onEvent(type, data) called by the publishing controller. 3. NotificationMessageFactory generates the appropriate message text for the event type. 4. NotificationRepository saves notification record with studentId, type, message, and timestamp. 5. Notification visible on next page load via GET /api/auth/me response.
Extensions	1a. Multiple events in quick succession — notifications batched. 1b. Student has no active session — notification persisted and shown on next login.
Patterns Applied	GRASP: Pure Fabrication (NotificationService, NotificationMessageFactory), Information Expert (NotificationRepository). GoF: Observer (BookingObserver interface implemented by NotificationService), Factory (NotificationMessageFactory). Implemented indirectly across all booking lifecycle flows.

UC-10: Manage Account Restriction

Field	Detail
Use Case ID	UC-10
Use Case Name	Manage Account Restriction
Primary Actor	Resource Manager, System (Automated)
Preconditions	For auto-restriction: student's noShowCount has reached the configured threshold (from UC-07). For manual review: Resource Manager is logged in.
Postconditions	Student account flagged as restricted. isRestricted=true, restrictionEndDate set. Student cannot submit booking requests for restriction duration.
Main Scenario	1. System detects violation count reached threshold (via UC-07 handler chain). 2. RestrictionHandler.handle() calls StudentRepository.applyRestriction(studentId, endDate). 3. Student account marked restricted with end date = today + restrictionThresholdDays. 4. NotificationHandler sends restriction notification. 5. Resource Manager can view restricted accounts via Penalty Dashboard. 6. Subsequent booking requests from restricted student return RESTRICTED status.
Extensions	1a. Student attempts booking while restricted — request blocked, RESTRICTED status returned. 1b. Restriction period expires — isRestricted automatically treated as false on next booking attempt.
Patterns Applied	GRASP: Information Expert (StudentRepository owns restriction data), Pure Fabrication (RestrictionHandler). GoF: Chain of Responsibility (RestrictionHandler is link 2 of the PenaltyHandler chain). Implemented indirectly as part of the UC-07 penalty handler chain.

UC-11: Register New Resource

Field	Detail
Use Case ID	UC-11
Use Case Name	Register New Resource

Field	Detail
Primary Actor	System Administrator or Resource Manager
Preconditions	User is logged in with ADMIN or MANAGER role. Room is physically available on campus.
Postconditions	New room stored in database. Room immediately visible in availability listings.
Main Scenario	1. User opens Register Resource page on admin/manager dashboard. 2. User fills in Room ID, Name, Building, Location, Capacity, and adds at least one time slot. 3. POST /api/rooms submitted. 4. ResourceValidator validates all fields (non-empty ID, capacity > 0, valid slot times). 5. RoomRepository checks for duplicate Room ID. 6. ResourceController creates new StudyRoom object (GRASP Creator). 7. RegisterResourceCommand.execute() persists room and time slots to database. 8. AuditLogRepository logs the registration action. 9. Confirmation shown; room appears in the Registered Rooms table below the form.
Extensions	1a. Duplicate Room ID — submission blocked, error message displayed. 1b. Required fields missing — field-level validation errors shown. 1c. No time slots provided — validation error shown.
Patterns Applied	GRASP: Controller (ResourceController), Creator (ResourceController creates Resource), Information Expert (RoomRepository), Pure Fabrication (ResourceValidator). GoF: Command (RegisterResourceCommand encapsulates INSERT + audit log).

UC-12: Verify Check-In

Field	Detail
Use Case ID	UC-12
Use Case Name	Verify Check-In
Primary Actor	Student
Preconditions	Student is logged in. Confirmed booking exists. Current time is within the check-in window.
Postconditions	Booking status updated to ACTIVE. Check-in timestamp recorded. Event logged.
Main Scenario	1. Student opens Check In tab on the student dashboard. 2. System calls GET /api/checkin/available to retrieve bookings within the check-in window. 3. Student clicks Check In on their booking. 4. CheckInController.requestCheckIn(bookingId) executes the fixed Template Method sequence: a. verifyOwnership(studentId) — confirmed on :Booking object. b. verifyTimeWindow() — window validated on :Booking object. c. updateStatus(ACTIVE) — status updated on :Booking object. d. recordTimestamp(bookingId) — timestamp written to BookingRepository. 5. ActivityLog entry created.
Extensions	1a. Check-in window not yet open — booking not shown in available list. 1b. Check-in window has elapsed — booking detected as no-show, UC-07 triggered. 1c. Student attempts to check in to another student's booking — denied.

Field	Detail
Patterns Applied	GRASP: Controller (CheckInController), Information Expert (:Booking knows its own ownership, status, and window). GoF: Template Method (fixed sequence: verifyOwnership → verifyTimeWindow → updateStatus → recordTimestamp enforced in CheckInController).

UC-13: Configure Fairness Policy

Field	Detail
Use Case ID	UC-13
Use Case Name	Configure Fairness Policy
Primary Actor	Resource Manager
Preconditions	Resource Manager is logged in. Policy configuration interface is accessible.
Postconditions	Updated settings saved. All subsequent allocations use new settings.
Main Scenario	1. Manager opens Policy Configuration panel. 2. System displays current policy settings. 3. Manager updates formula weights, penalty value, or restriction threshold. 4. System validates all values are within permissible ranges. 5. New settings written to FairnessPolicies table. 6. Change logged with manager ID, timestamp, and old/new values.
Extensions	1a. Value outside permitted range — field highlighted, save blocked. 1b. All weights set to zero — configuration rejected.
Patterns Applied	GRASP: Controller (PolicyController), Information Expert (FairnessPolicyRepository), Pure Fabrication (PolicyValidator). Not implemented — policy table exists in DB and is read by the allocation engine, but no UI or API endpoint for modification was built within the project timeline.

UC-14: Switch Allocation Mode

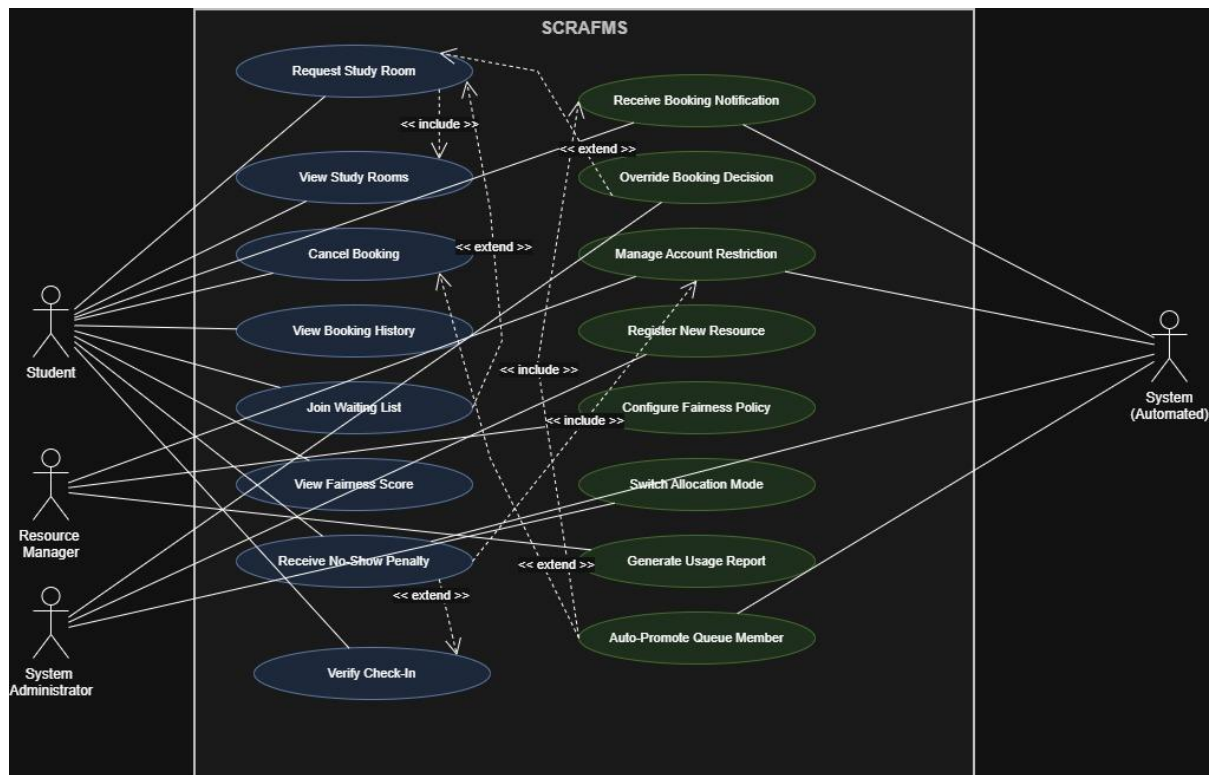
Field	Detail
Use Case ID	UC-14
Use Case Name	Switch Allocation Mode
Primary Actor	System Administrator
Preconditions	Administrator is logged in. Desired mode differs from the current active mode.
Postconditions	Selected mode saved as active. All subsequent booking requests processed using the new strategy.
Main Scenario	1. Admin opens System Config page on the admin dashboard. 2. System calls GET /api/allocation/mode to retrieve current mode from AllocationModeRepository. 3. Admin clicks the desired mode card (Fair Use, FCFS, or Exam Mode). 4. Admin clicks Apply Mode. System checks mode differs from current. 5. POST /api/allocation/mode

Field	Detail
	submitted. 6. AllocationModeRepository.saveActiveMode(newMode) persists the change. 7. AuditLogRepository logs the mode switch with adminId and timestamp. 8. AllocationService re-reads active mode from DB on the next allocate() call, injecting the correct strategy automatically. 9. Confirmation displayed; current mode badge updates on screen.
Extensions	1a. Same mode selected — Apply Mode button remains disabled. 1b. Exam Mode selected — all subsequent booking requests return EXAM_BLOCKED; no booking records inserted.
Patterns Applied	GRASP: Controller (AllocationController), Information Expert (AllocationModeRepository), Pure Fabrication (AuditLogRepository). GoF: Strategy (AllocationService re-reads mode from DB on every allocate() call — FairUseStrategy, FCFSStrategy, or ExamModeStrategy injected at runtime without modifying AllocationService).

UC-15: Auto-Promote Queue Member

Field	Detail
Use Case ID	UC-15
Use Case Name	Auto-Promote Queue Member
Primary Actor	System (Automated)
Preconditions	A confirmed booking has been cancelled (UC-MR-03) or marked NO_SHOW (UC-07). At least one student is in the waiting queue for the released slot.
Postconditions	Released slot assigned to highest-priority eligible queued student. Queue entry removed. Student notified. New booking record created.
Main Scenario	1. Booking cancelled or marked NO_SHOW — slot freed via RoomRepository. 2. WaitingQueueService.promoteNext(slotId) called. 3. WaitingQueueRepository retrieves queue entries ordered by priorityScore DESC (Fair Use) or requestedAt ASC (FCFS). 4. If queue is empty: slot marked available, return. 5. Top entry checked — if student is restricted, entry skipped, next checked. 6. New CONFIRMED booking created for promoted student via BookingRepository. 7. Promoted queue entry deleted from WaitingQueueEntries. 8. NotificationService sends promotion notification to student. 9. ActivityLog entry created.
Extensions	1a. Queue empty when slot released — slot marked available, no promotion performed. 1b. Top student is restricted — entry skipped, next eligible student promoted. 1c. All queued students are restricted — slot marked available, no promotion.
Patterns Applied	GRASP: Controller (WaitingListController), Information Expert (WaitingQueueRepository, BookingRepository), Pure Fabrication (WaitingQueueService). GoF: Observer (SLOT_RELEASED event triggers NotificationService), Strategy (queue ordering changes based on active allocation mode).

2.6 Use Case Diagram



3. Other Non-Functional Requirements

3.1 Performance Requirements

Allocation decisions must be processed and returned to the student within 3 seconds of submitting a booking request. The AllocationService performs at most three sequential database queries (eligibility check, fairness score read, booking INSERT or queue INSERT) and all queries use indexed primary key lookups via prepared statements. In testing, allocation decisions consistently returned in under 500 milliseconds on the development machine.

The web server uses a fixed thread pool of 4 threads to handle concurrent requests. All API endpoints must respond within 2 seconds for operations involving a single database query, and within 5 seconds for operations involving multiple queries (such as queue processing).

3.2 Safety Requirements

The no-show detection background thread is implemented as a daemon thread, ensuring it does not prevent application shutdown. All database operations use try-with-resources blocks to guarantee connection cleanup and prevent resource leaks. The thread-safe DatabaseConnection singleton with double-checked locking prevents concurrent connection conflicts between the web request thread pool and the background no-show thread.

3.3 Security Requirements

All passwords are hashed using SHA-256 via `java.security.MessageDigest` before storage and before comparison at login. Plain-text passwords never appear in the database, codebase, or network traffic. Web sessions are managed via server-side token maps (`ConcurrentHashMap`) with `HttpOnly` cookies. Session tokens are UUID v4 strings generated by `java.util.UUID.randomUUID()`. All SQL queries use prepared statements with parameterised inputs, eliminating SQL injection risk.

3.4 Software Quality Attributes

- **Maintainability** — strict layered architecture ensures each layer can be modified independently. Repository classes can be swapped without touching service or controller logic.
- **Extensibility** — GoF Strategy pattern allows new allocation algorithms to be added as new concrete strategy classes without modifying AllocationService. Chain of Responsibility allows new penalty handling steps to be added without modifying existing handlers.
- **Reliability** — all repository methods include exception handling with console error logging. The fallback login mechanism in AuthService ensures the system remains accessible even if JDBC connectivity is temporarily interrupted.
- **Usability** — the web interface provides immediate visual feedback through toast notifications, status badges, and loading indicators. Empty states and error messages guide users when no data is available.
- **Auditability** — all administrative actions (overrides, mode switches) are written to immutable audit log tables with administrator ID, timestamp, and reason. No-show violations are logged to a separate violation log.

3.5 Business Rules

- A student may not join the waiting queue for a slot they already hold a confirmed booking for
- A student may not hold more bookings per week than the configured weekly usage limit
- An override reason must be between 10 and 500 characters — enforced at both UI and service layers
- Account restrictions are applied automatically when noShowCount reaches the configured threshold and are lifted automatically when the restriction period expires
- In Exam Mode, all new booking requests return EXAM_BLOCKED and no booking records are inserted into the database
- All allocation mode switches take immediate effect — subsequent requests use the new strategy regardless of when the server was started

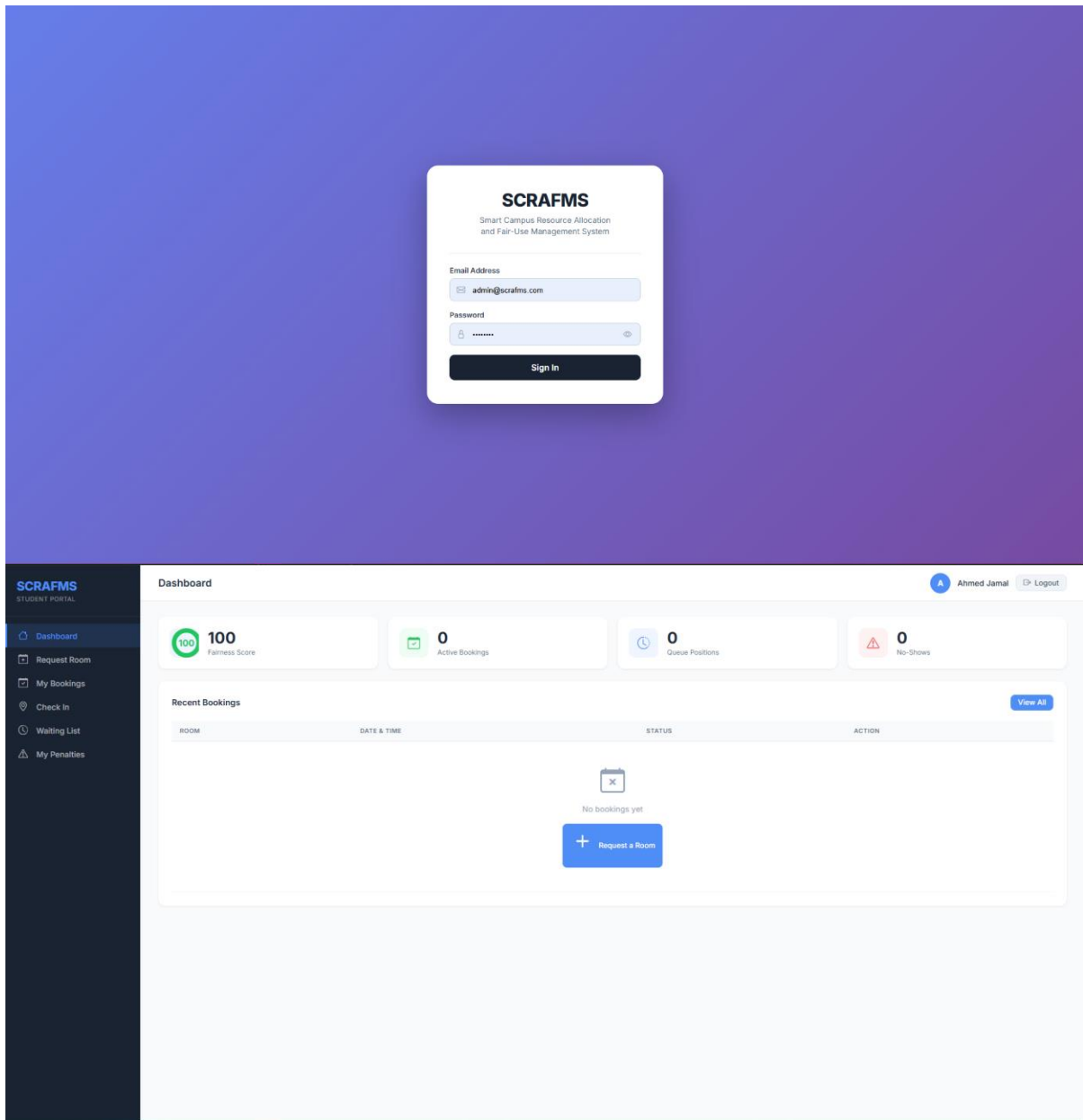
3.6 Operating Environment

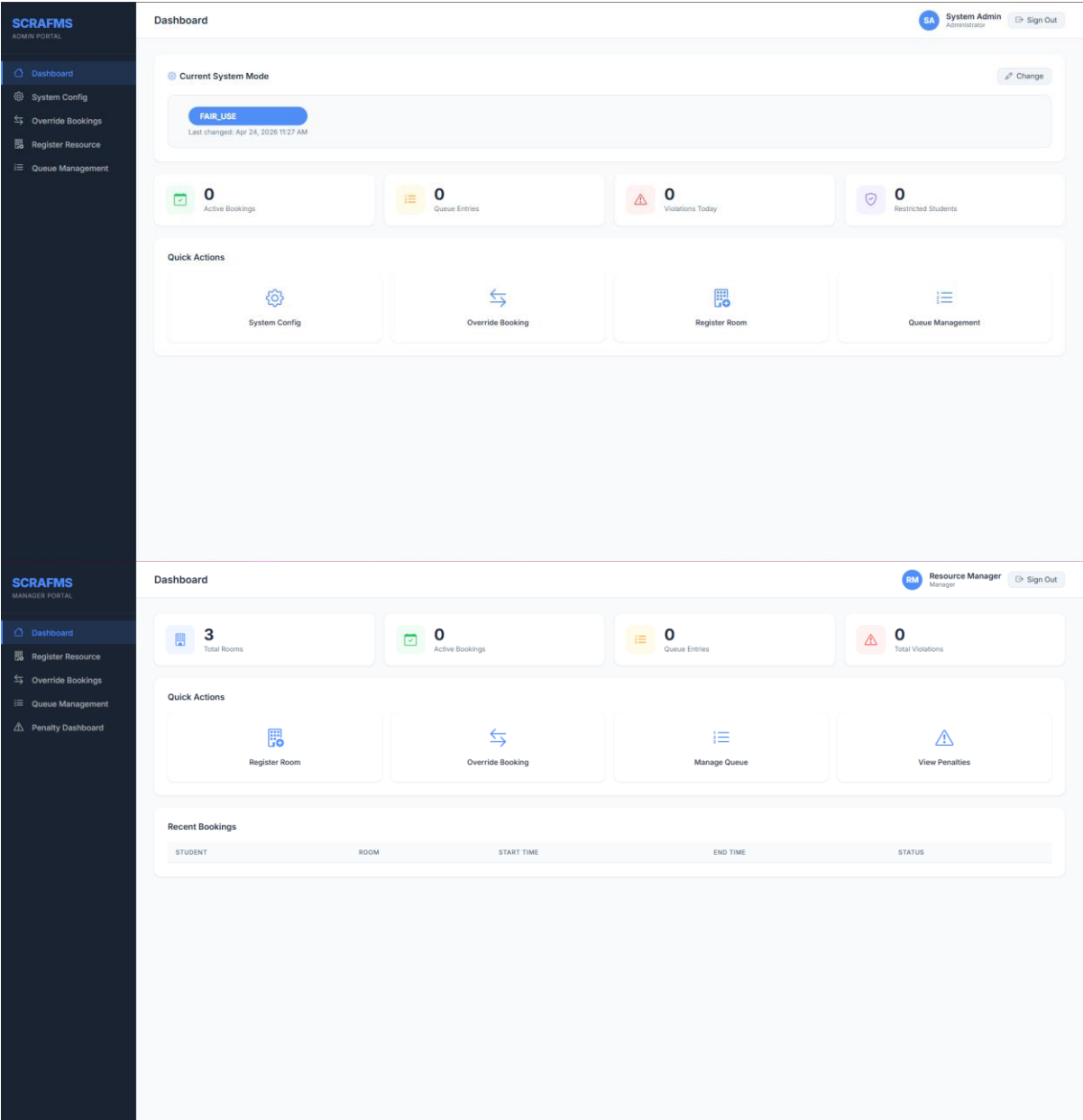
- Operating System: Windows 10/11 (64-bit)
- Java Runtime: OpenJDK 21 (Temurin distribution)
- Database: SQL Server Express (local instance)
- Build Tool: Apache Maven 3.9.x
- Web Browser: Any modern browser supporting ES6 (Chrome, Firefox, Edge)
- Network: Localhost only (port 8080) — no external network access required
- JDBC Driver: Microsoft JDBC Driver for SQL Server 12.6.1

3.7 User Interfaces

The user interface is a web application served by the embedded Java HttpServer on localhost:8080. Five HTML pages are provided with a shared CSS stylesheet and JavaScript utility library. Navigation between screens occurs without full page reloads using dynamic content injection.

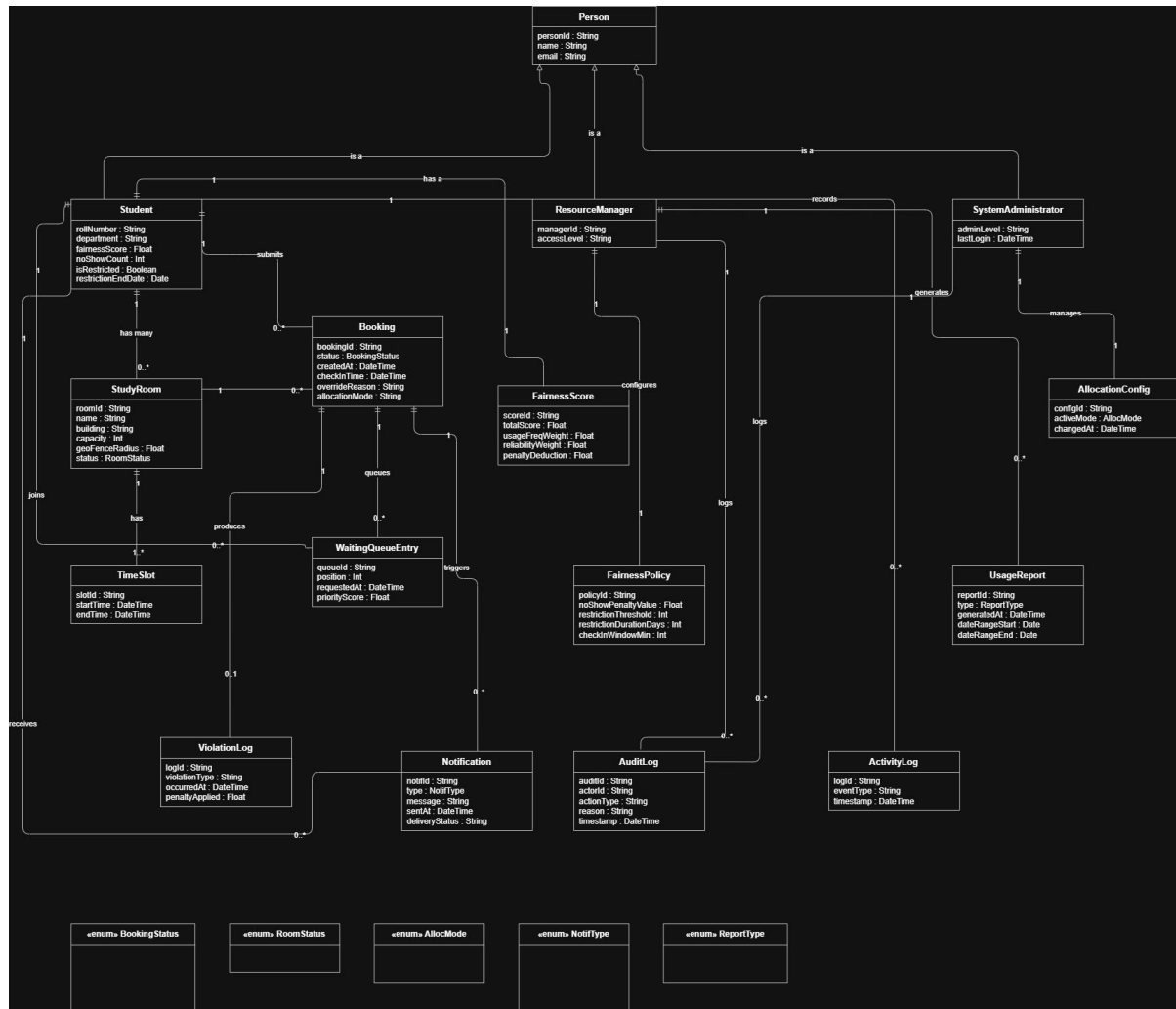
- Login Page — standalone centred card with email and password fields, role-based redirect on success
- Student Dashboard — single-page app with sidebar navigation: Request Room, My Bookings, Check In, Waiting List, My Penalties
- Manager Dashboard — sidebar navigation: Register Resource, Override Bookings, Queue Management, Penalty Dashboard
- Admin Dashboard — sidebar navigation: System Config (mode switching), Override Bookings, Register Resource, Queue Management





4. Domain Model

The domain model captures the key conceptual entities of SCRAFMS and their relationships. It was developed iteratively across Deliverables 2 and 3. The version below reflects the final implemented database schema.



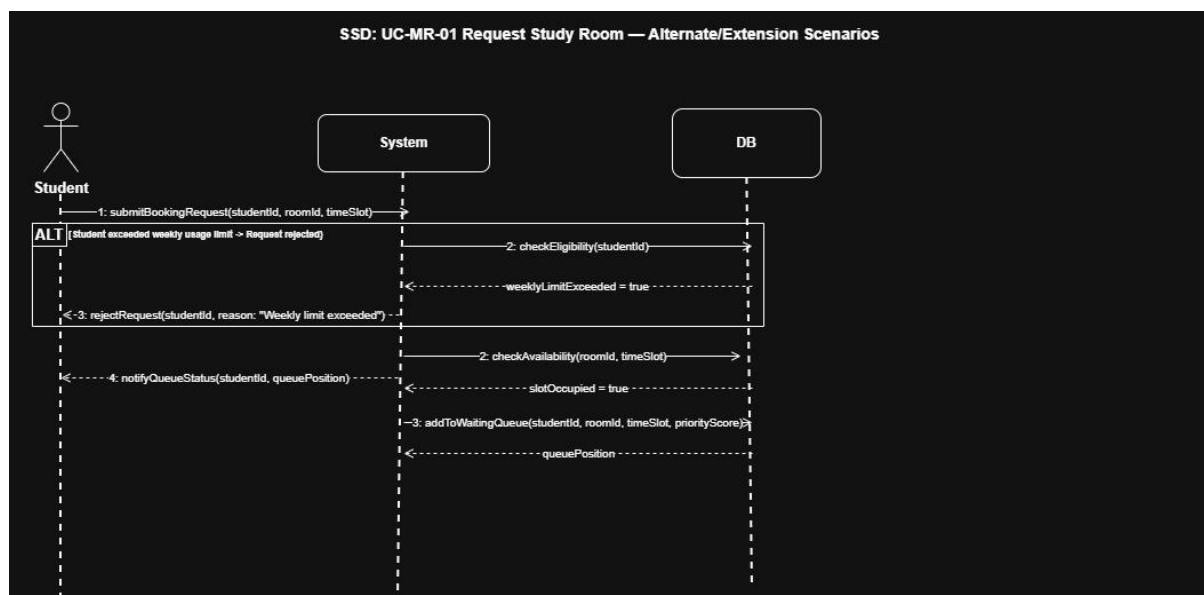
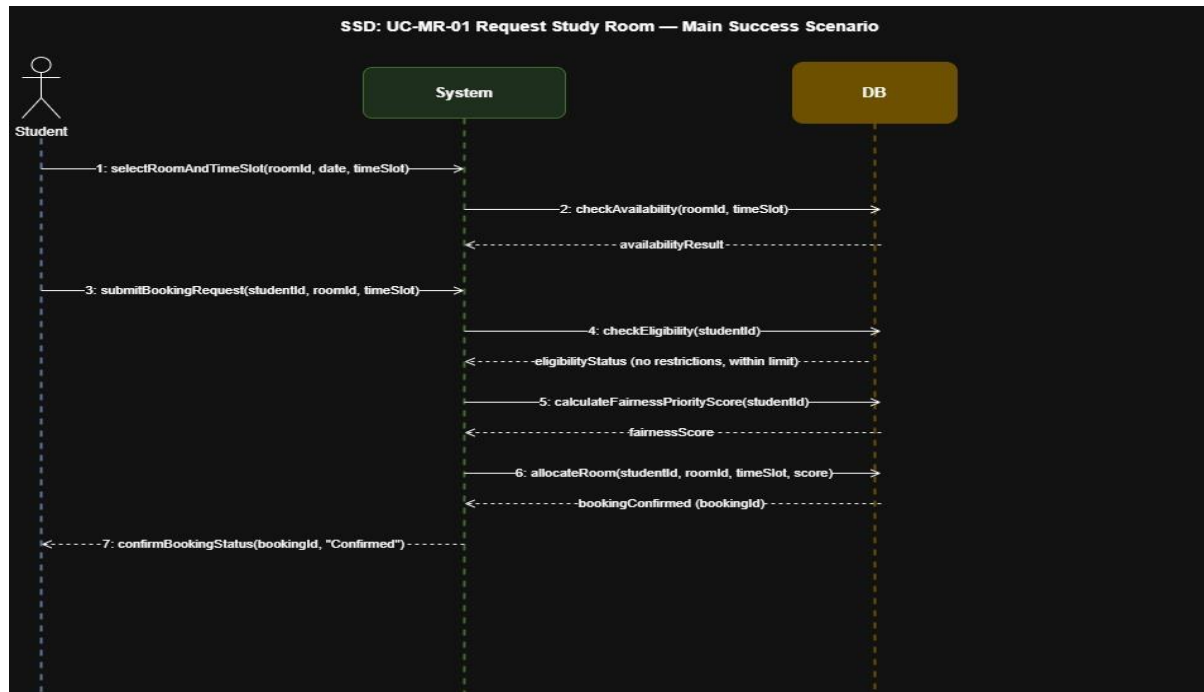
Entity	Description
Person	Base entity for all users. Subtyped into Student, ResourceManager, SystemAdministrator.
Student	Holds fairnessScore, noShowCount, isRestricted, and restrictionEndDate.
ResourceManager	University staff responsible for resource management and policy configuration.
SystemAdministrator	Technical personnel for allocation strategy management and administrative overrides.
Booking	Records a student's room reservation with status, start/end times, check-in time, and allocationMode.
StudyRoom	Physical room with capacity, building, location, and geofenceRadius.

Entity	Description
TimeSlot	Specific availability window for a room with startTime, endTime, and isAvailable flag.
WaitingQueueEntry	A student's position in the queue for a specific slot with priorityScore and requestedAt.
FairnessPolicy	Configurable parameters: penaltyValue, restrictionThreshold, checkInWindowMinutes, formula weights.
AllocationConfig	Stores the currently active allocation mode (FAIR_USE, FCFS, EXAM_MODE) and changedAt timestamp.
ViolationLog	Immutable record of each no-show penalty applied to a student.
AuditLog	Immutable record of all administrative actions (overrides, mode switches).
ActivityLog	Operational event log for cancellations, check-ins, and auto-promotions.
Notification	In-app notification records delivered to students at booking lifecycle events.

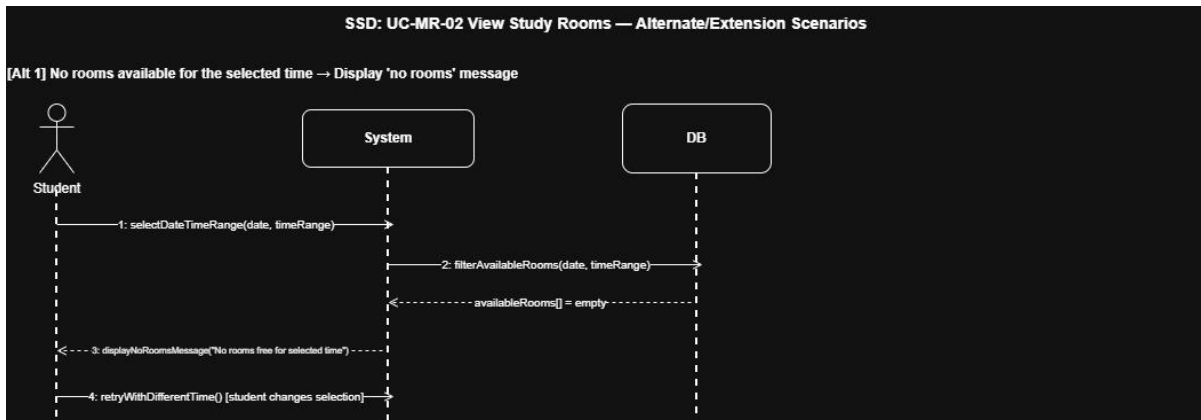
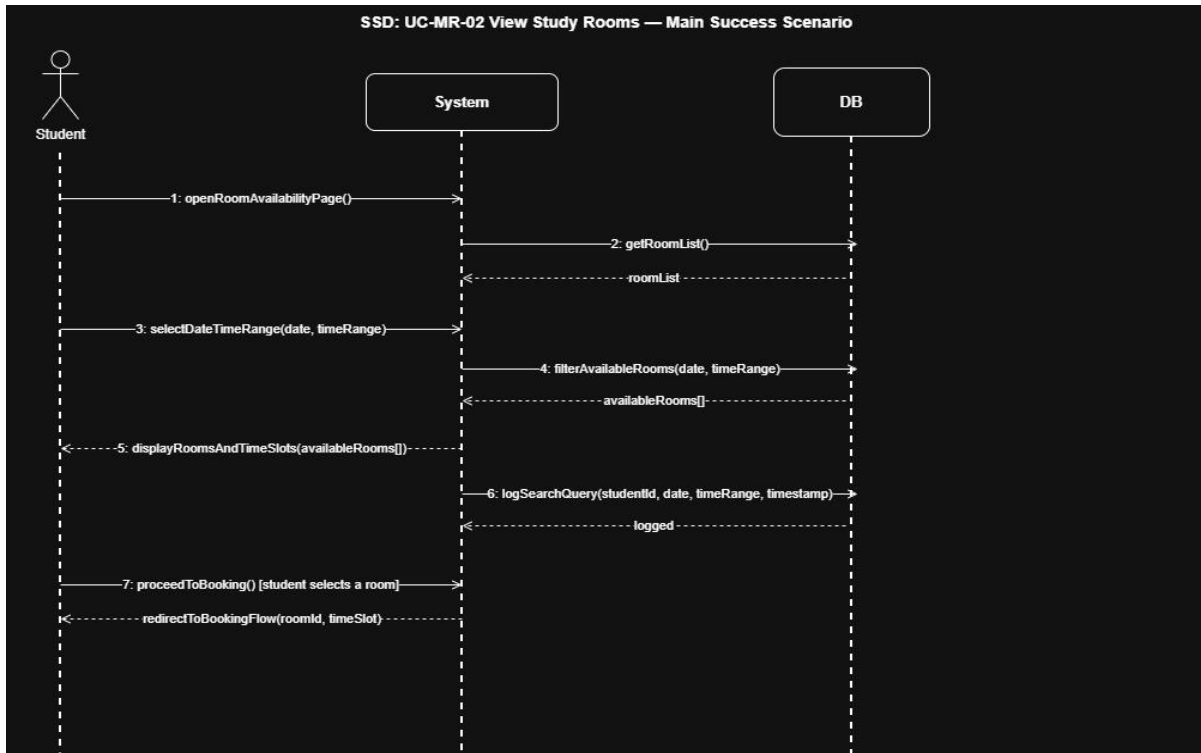
5. System Sequence Diagrams

System Sequence Diagrams capture the interactions between actors and the system boundary for each use case's main success scenario. Each SSD shows the external messages exchanged and the system's responses without revealing internal implementation details.

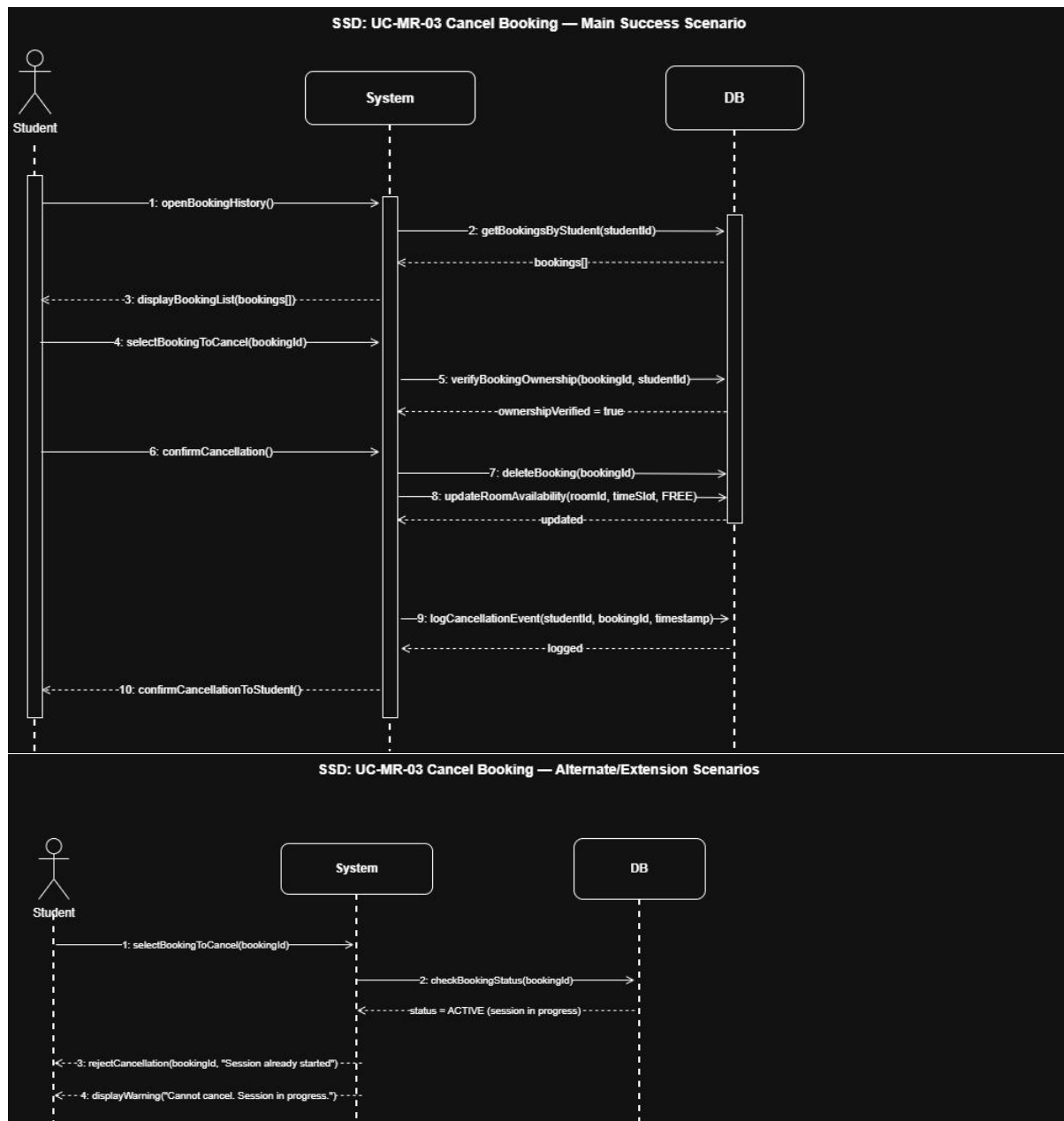
UC-MR-01: Request Study Room



UC-MR-02: View Study Rooms



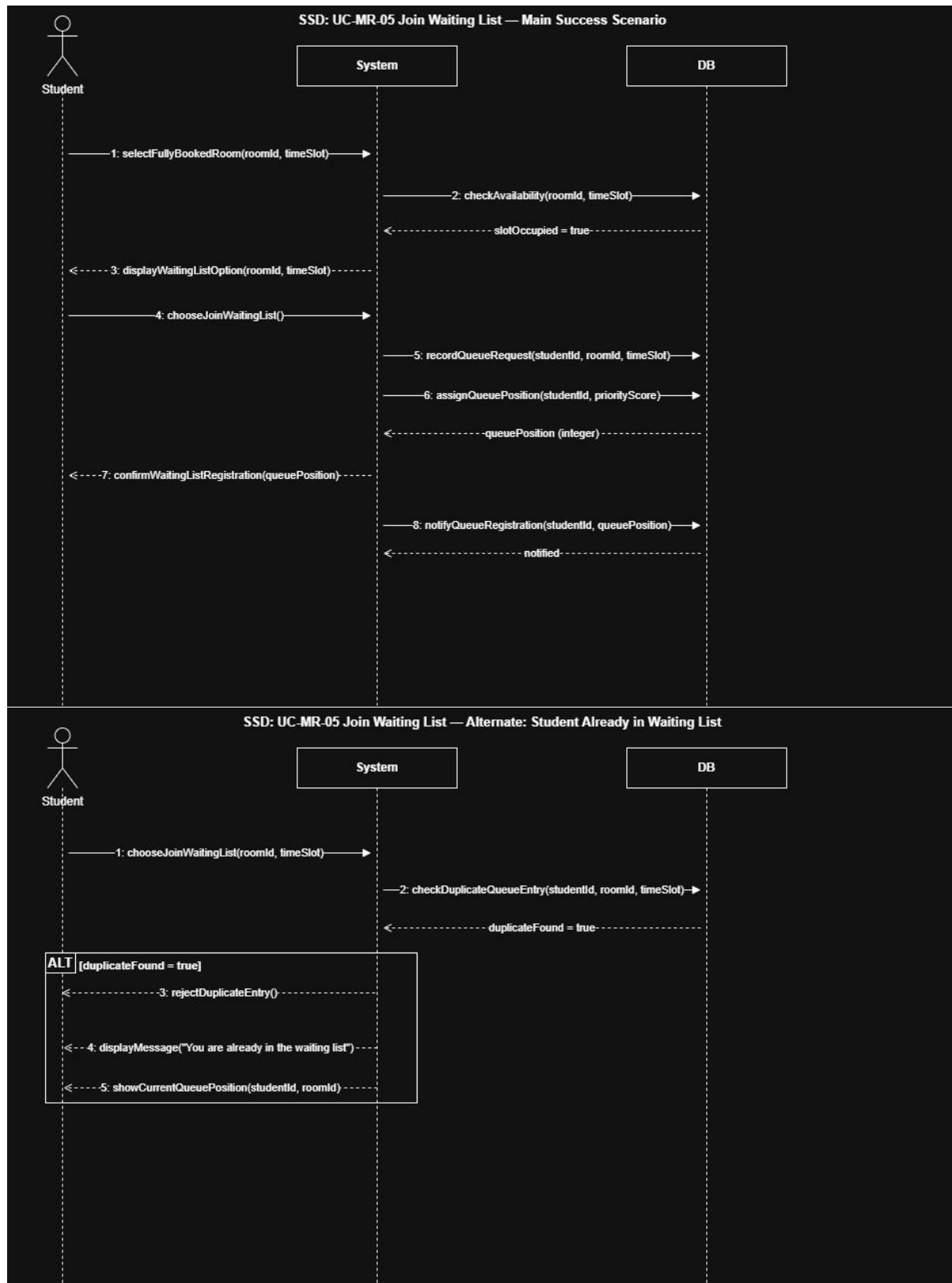
UC-MR-03: Cancel Booking



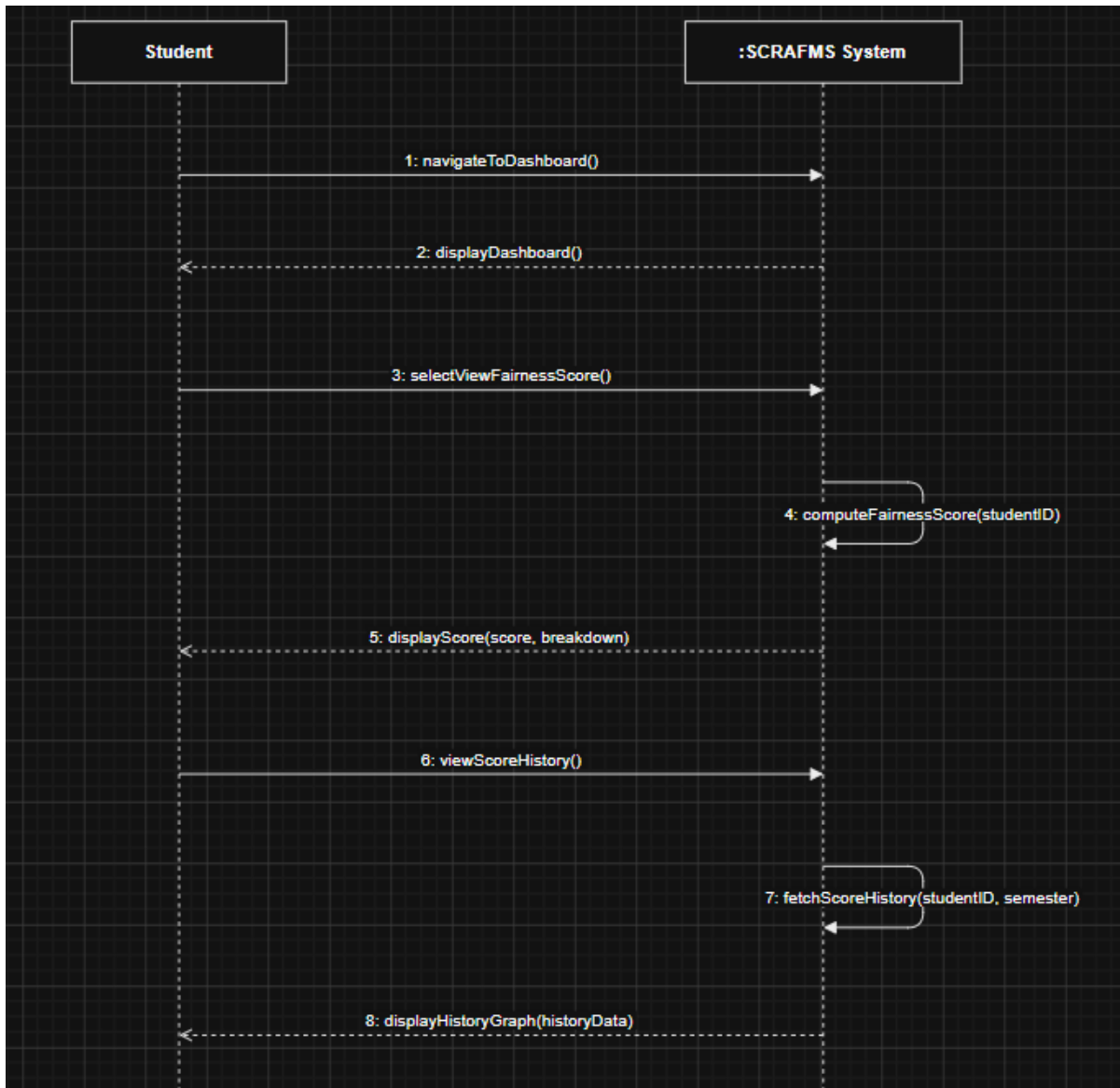
UC-MR-04: View Booking History



UC-MR-05: Join Waiting List

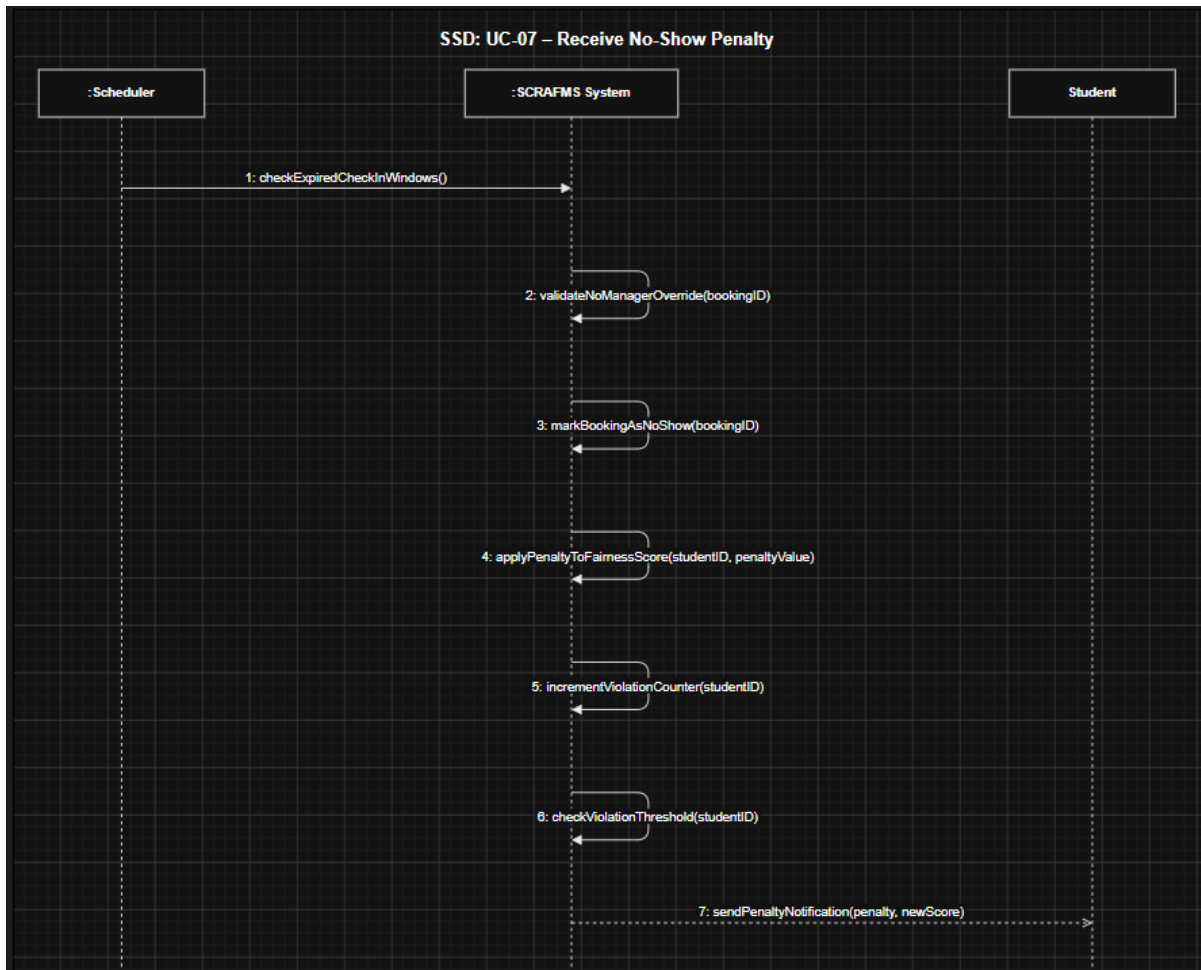


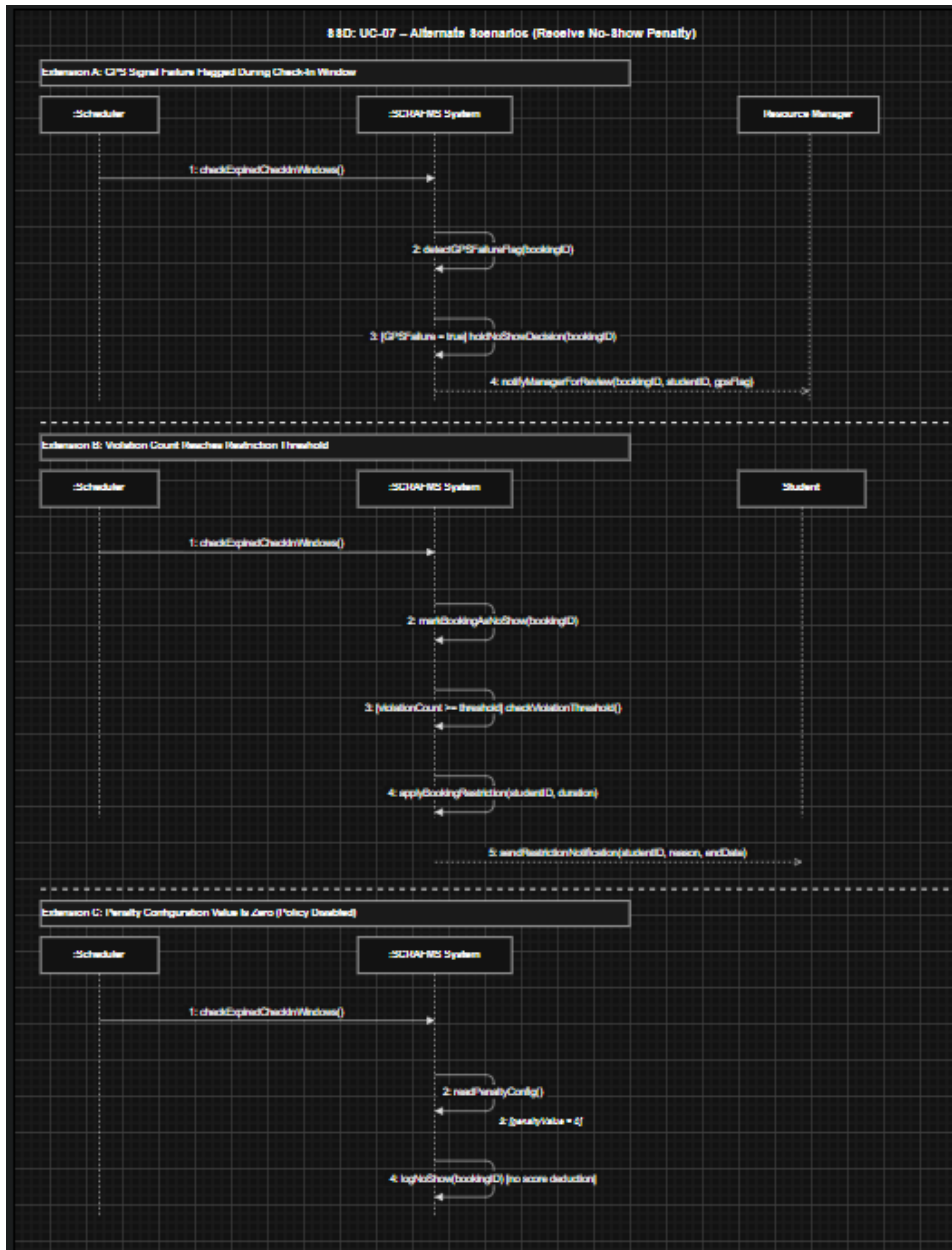
UC-06: View Fairness Score



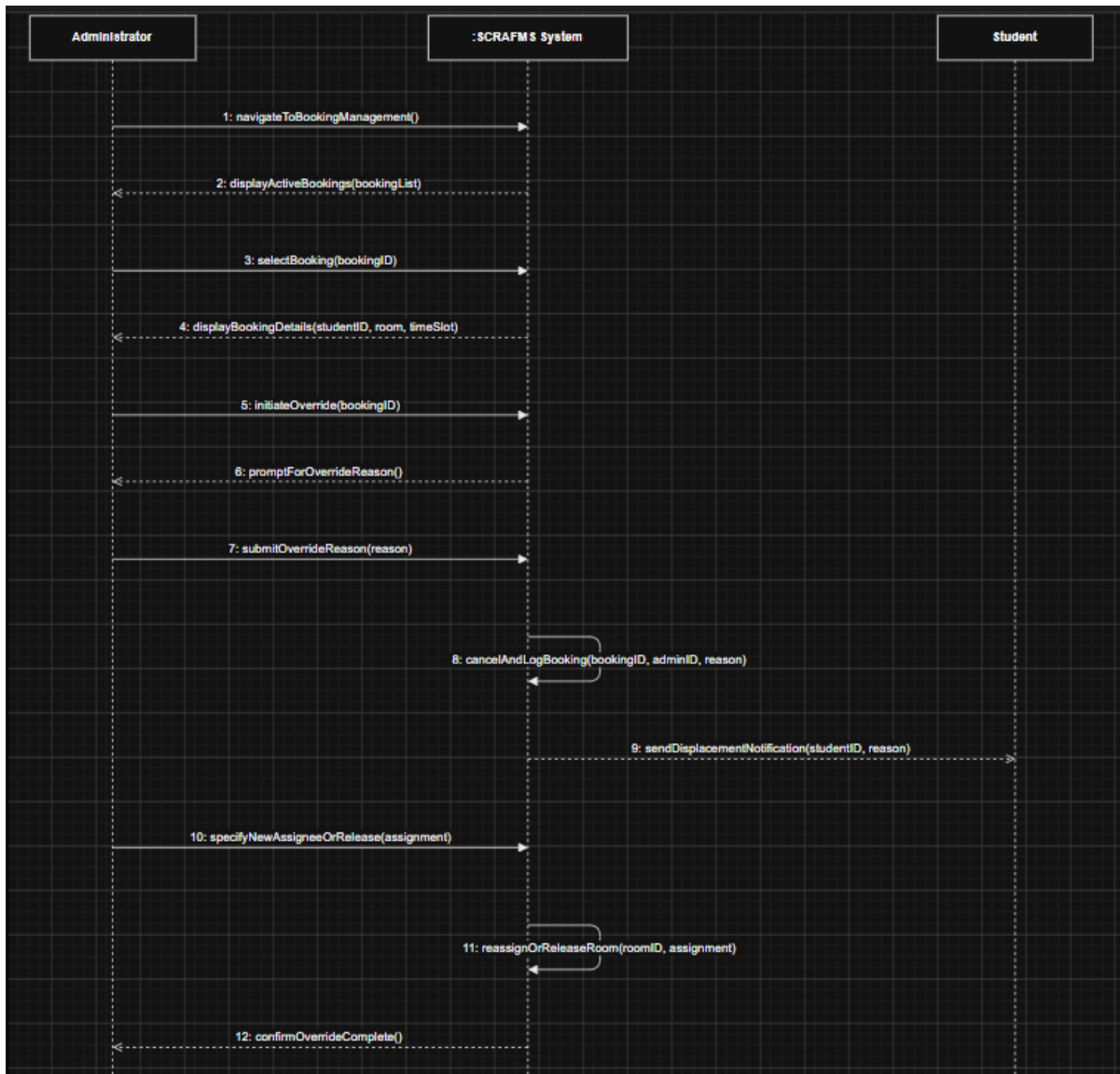


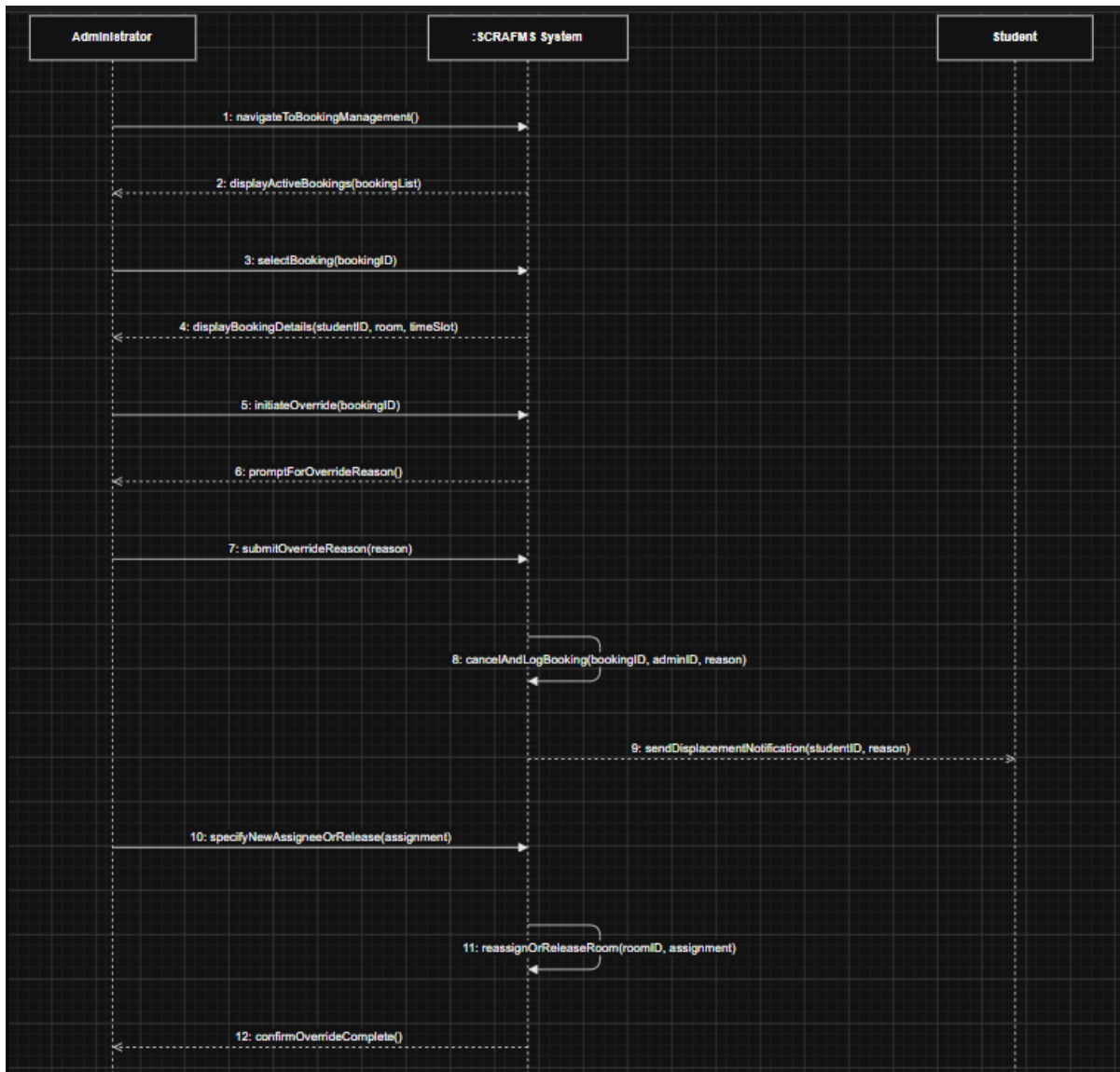
UC-07: Receive No-Show Penalty





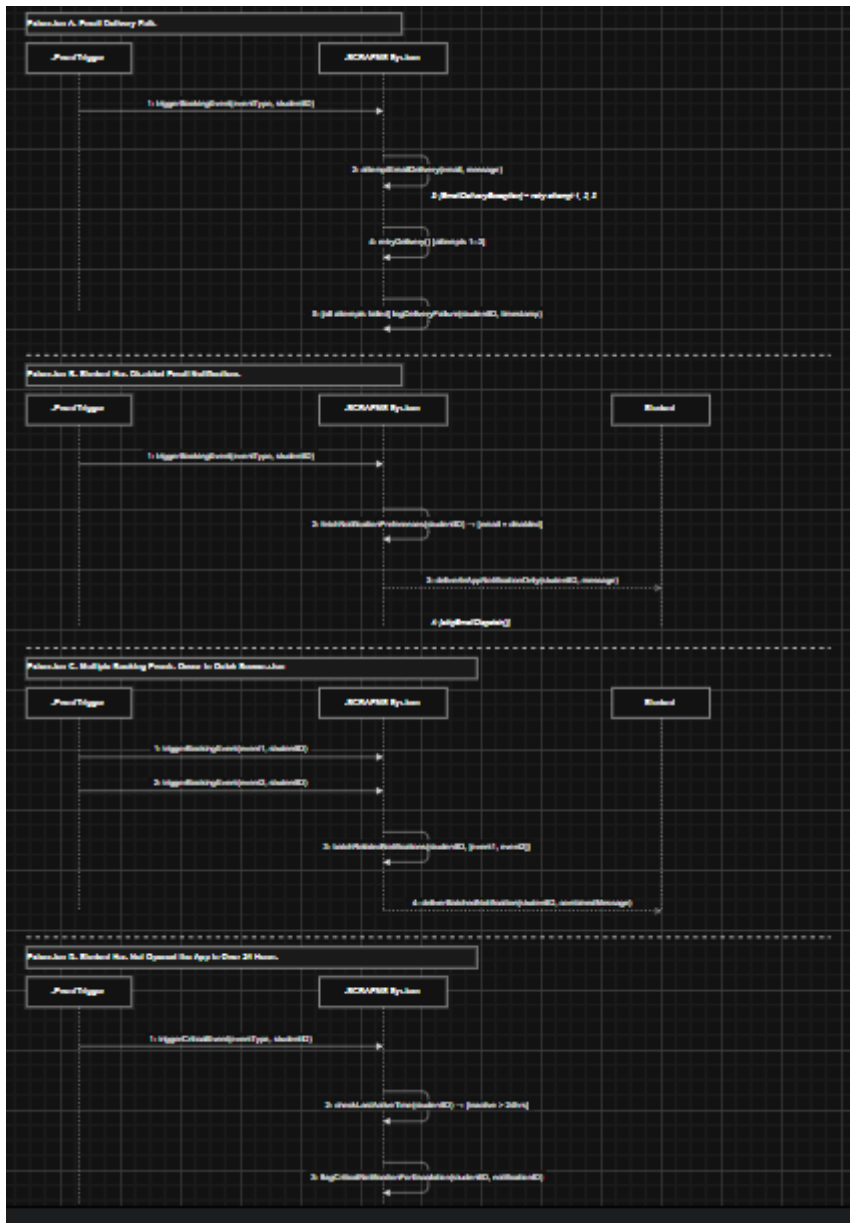
UC-08: Override Booking Decision



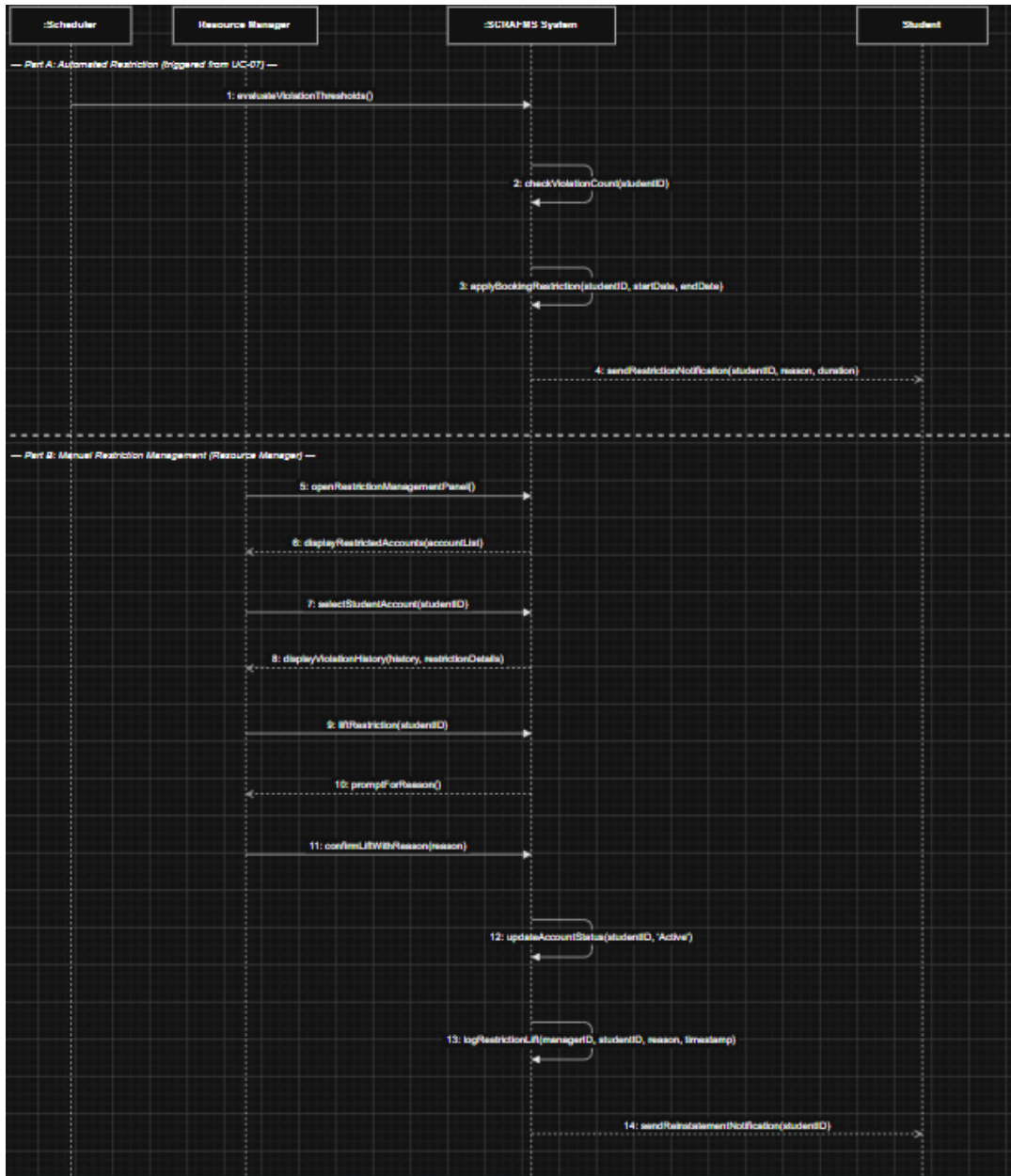


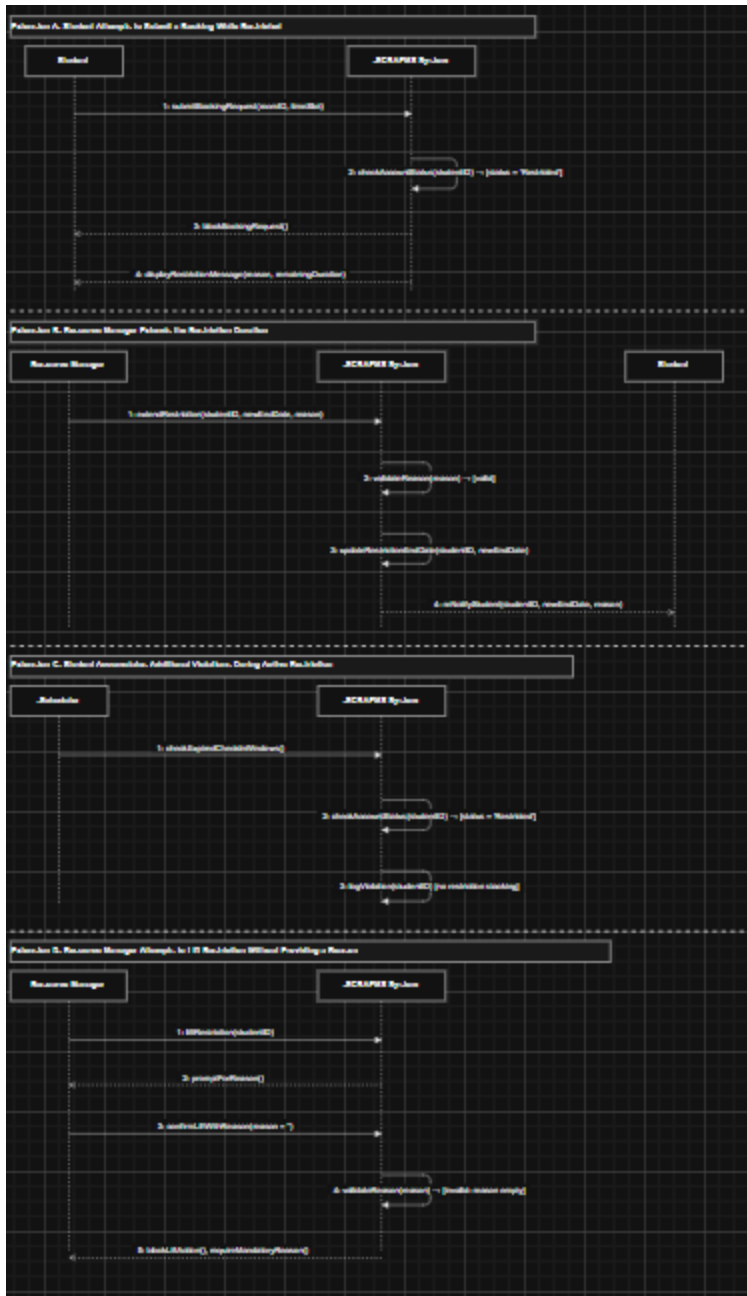
UC-09: Receive Booking Notification



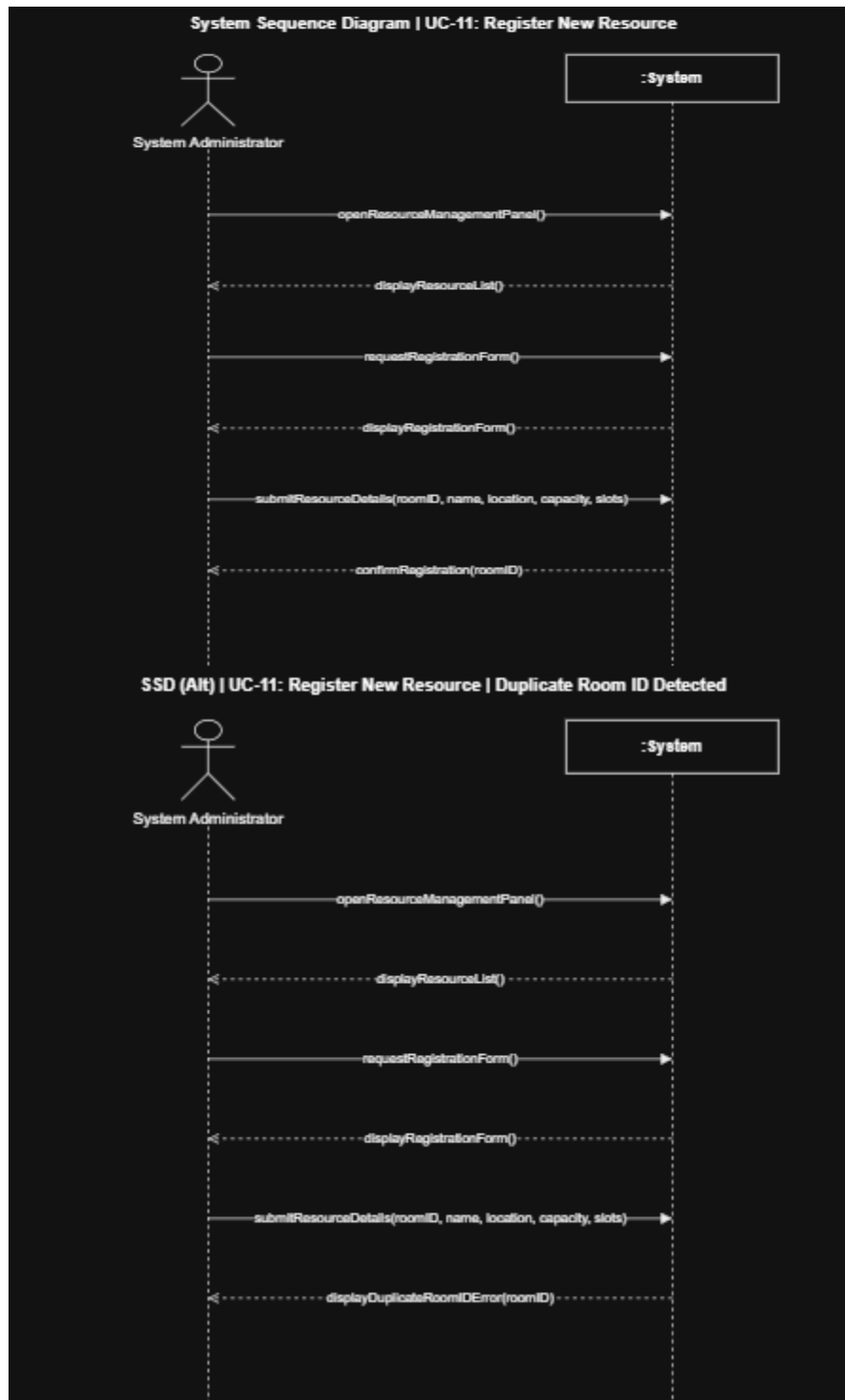


UC-10: Manage Account Restriction

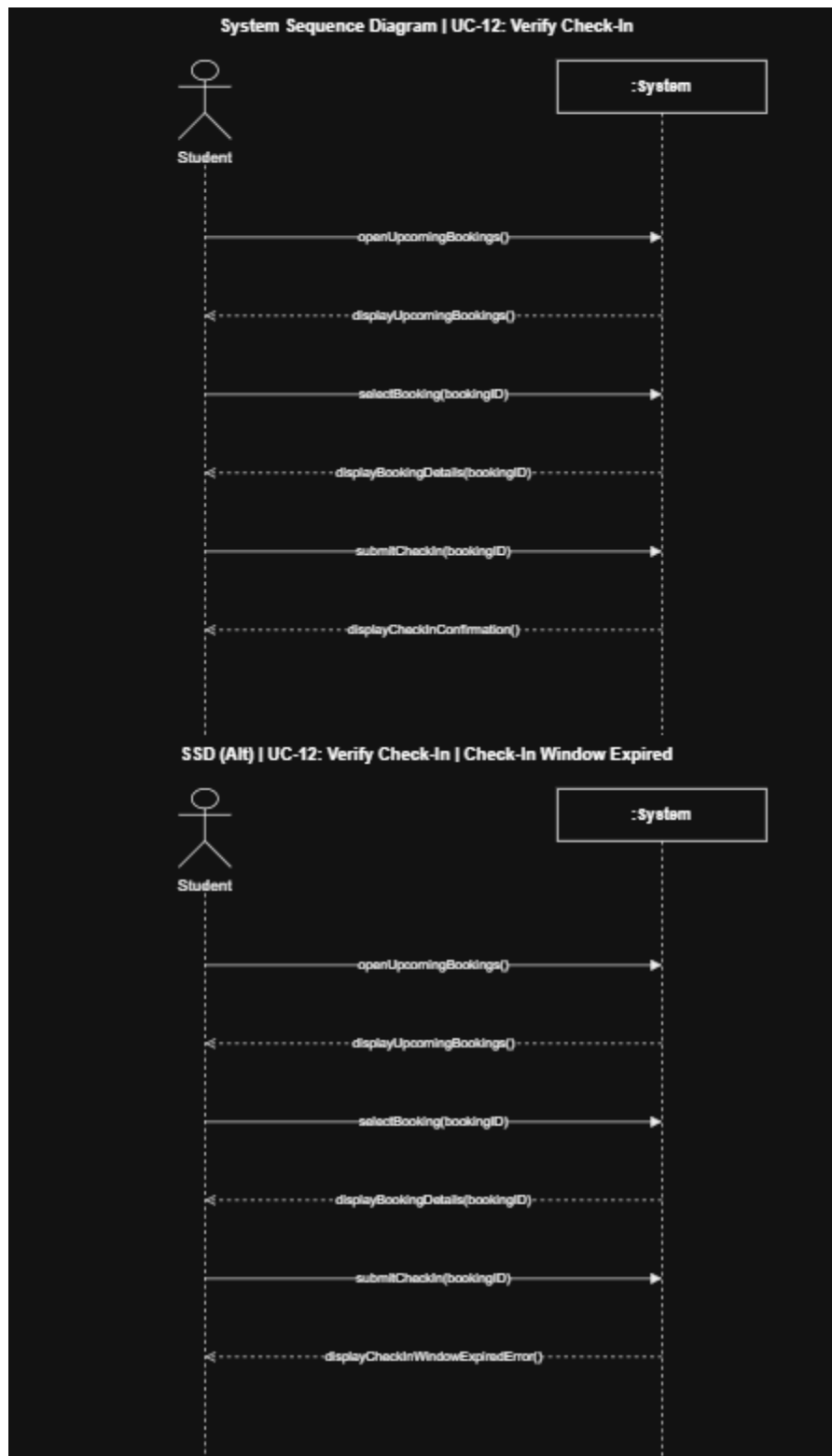




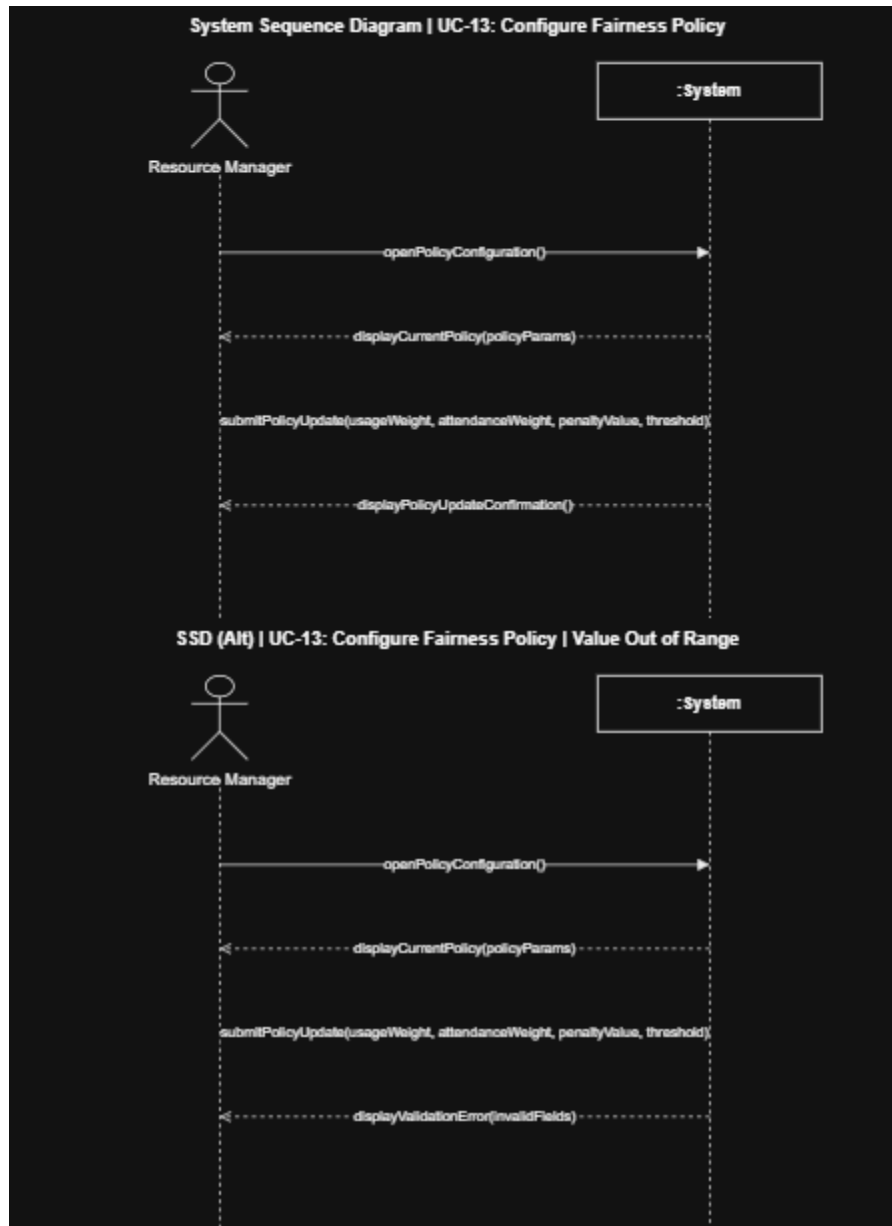
UC-11: Register New Resource



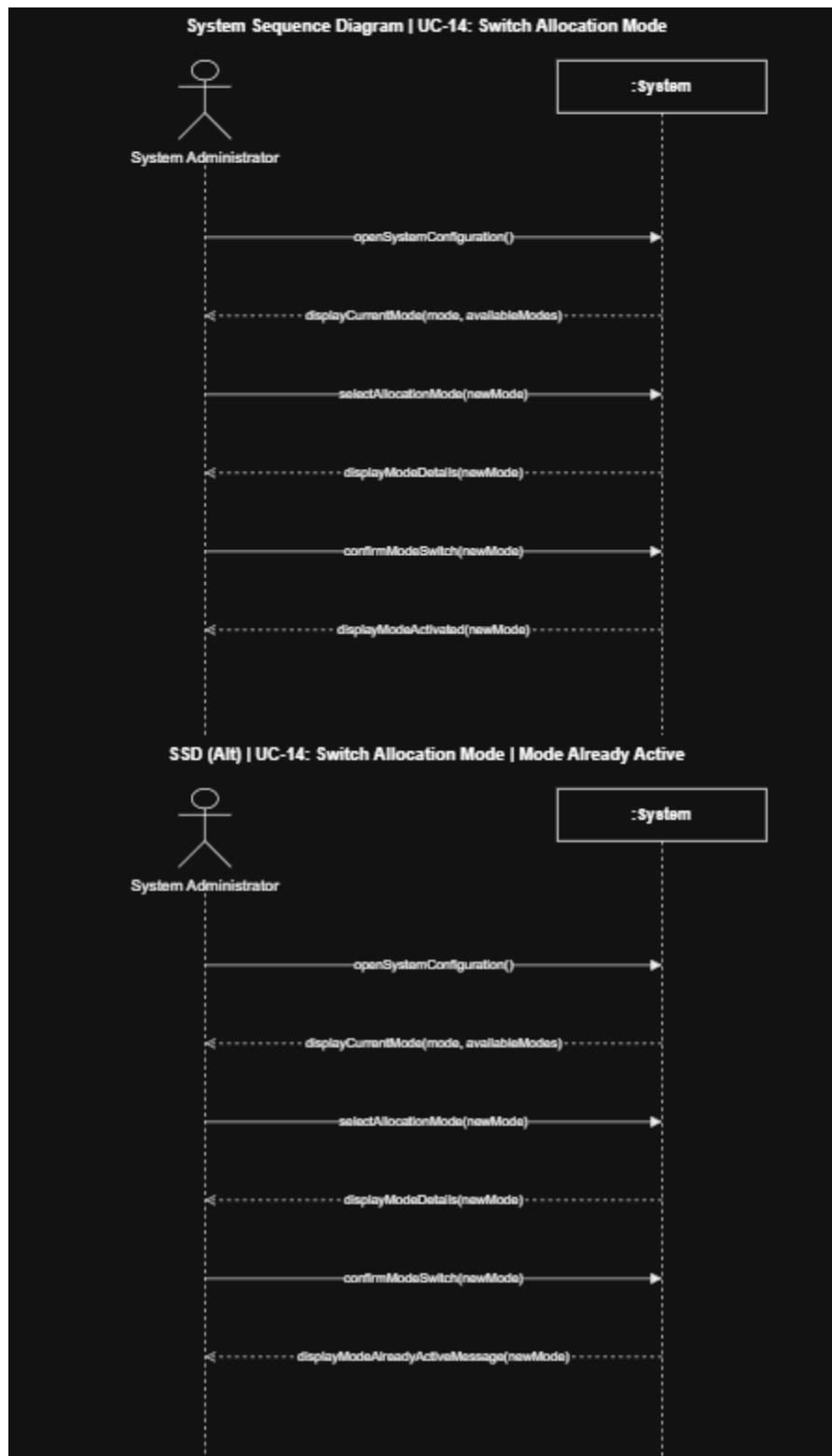
UC-12: Verify Check-In



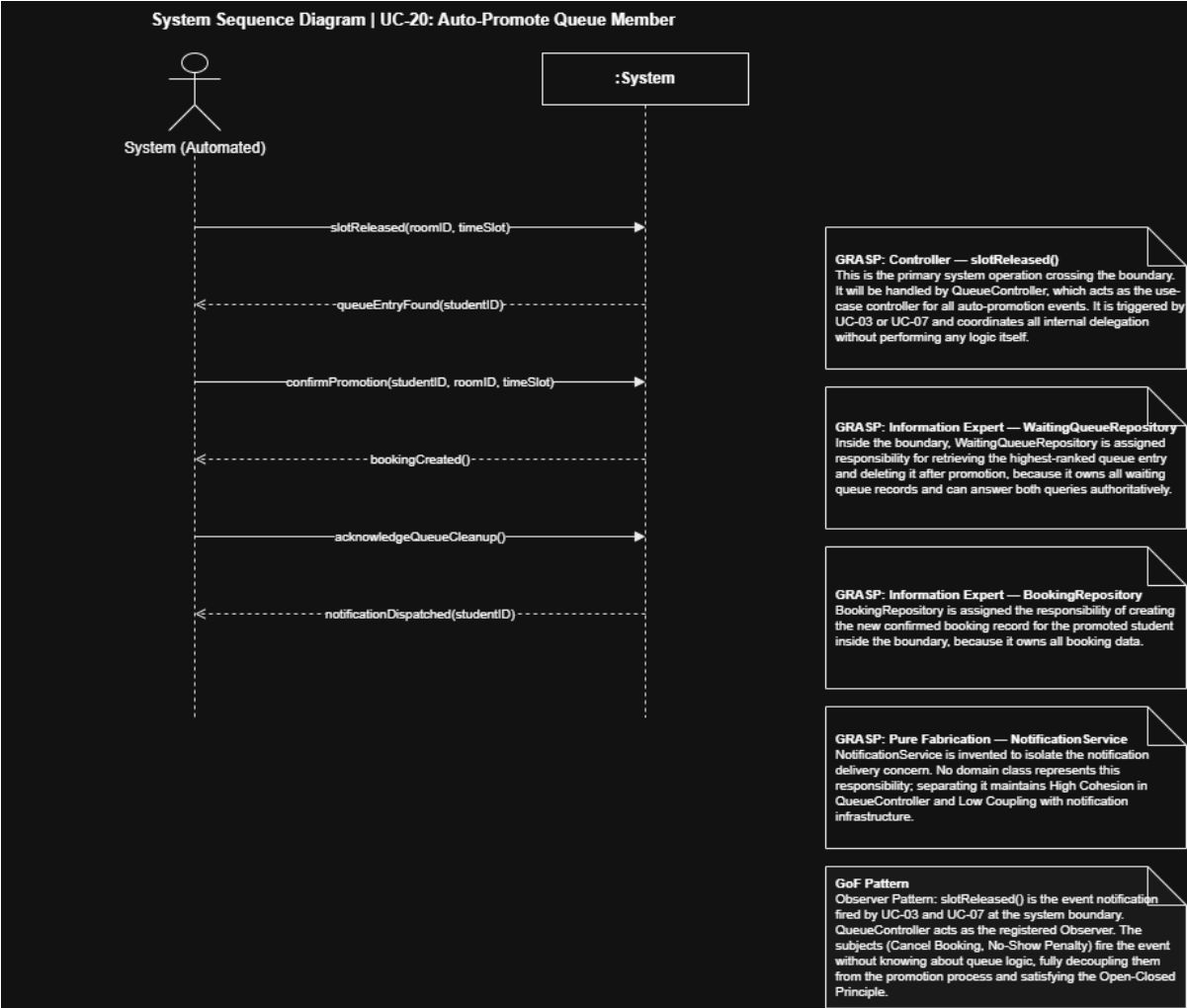
UC-13: Configure Fairness Policy

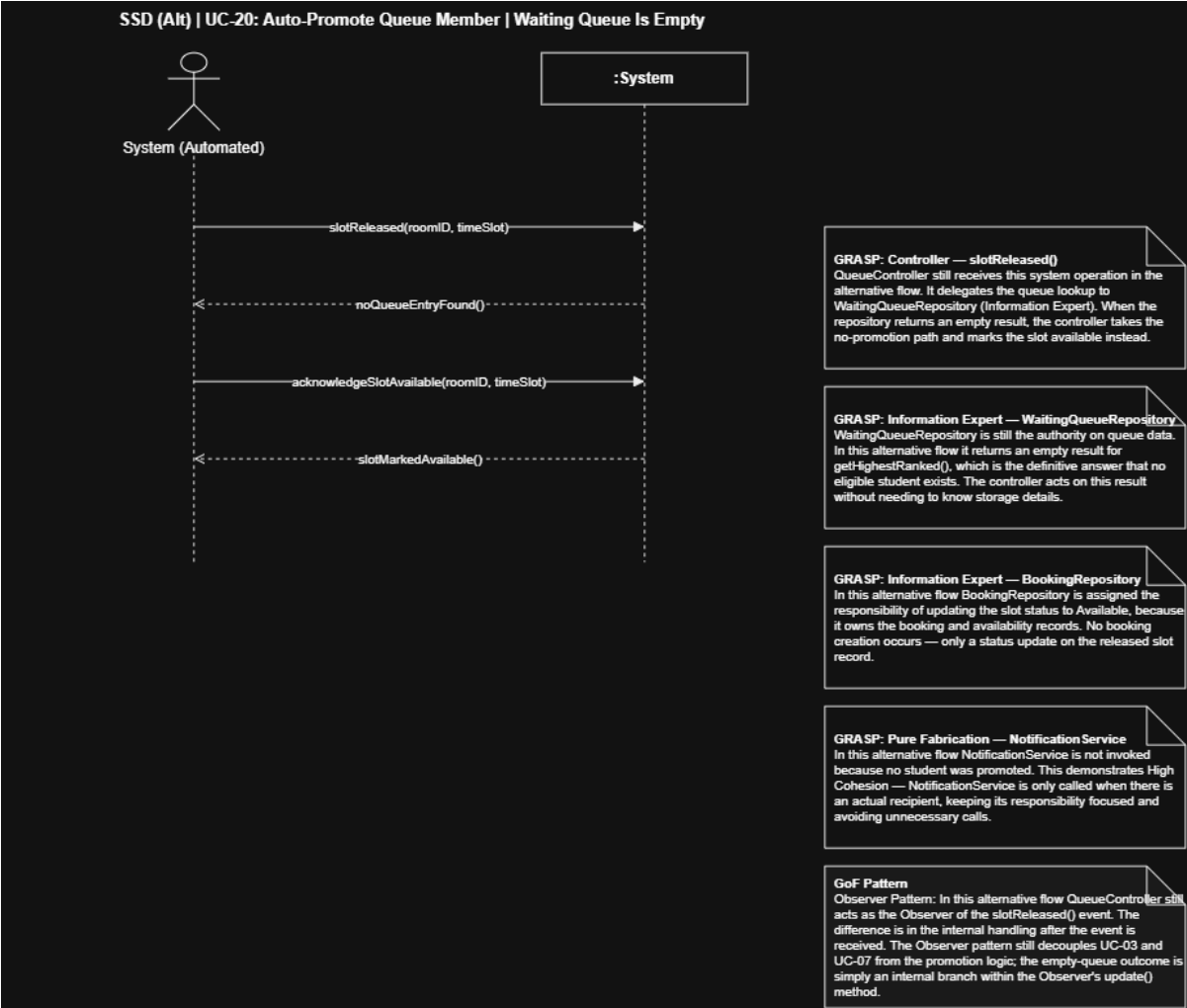


UC-14: Switch Allocation Mode



UC-15: Auto-Promote Queue Member

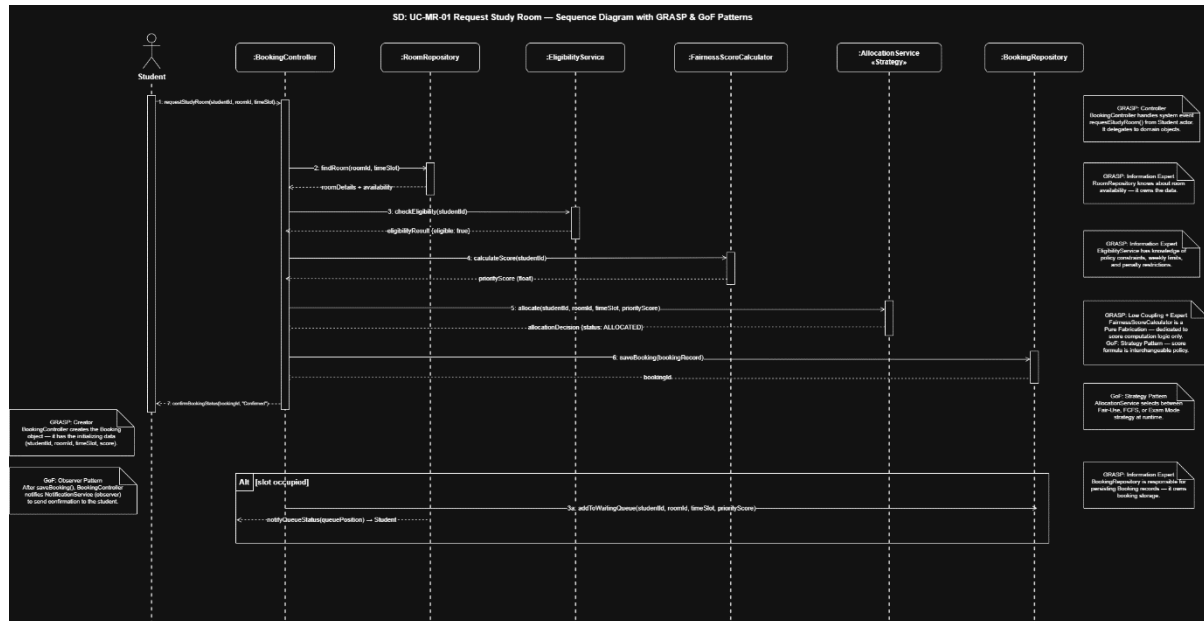




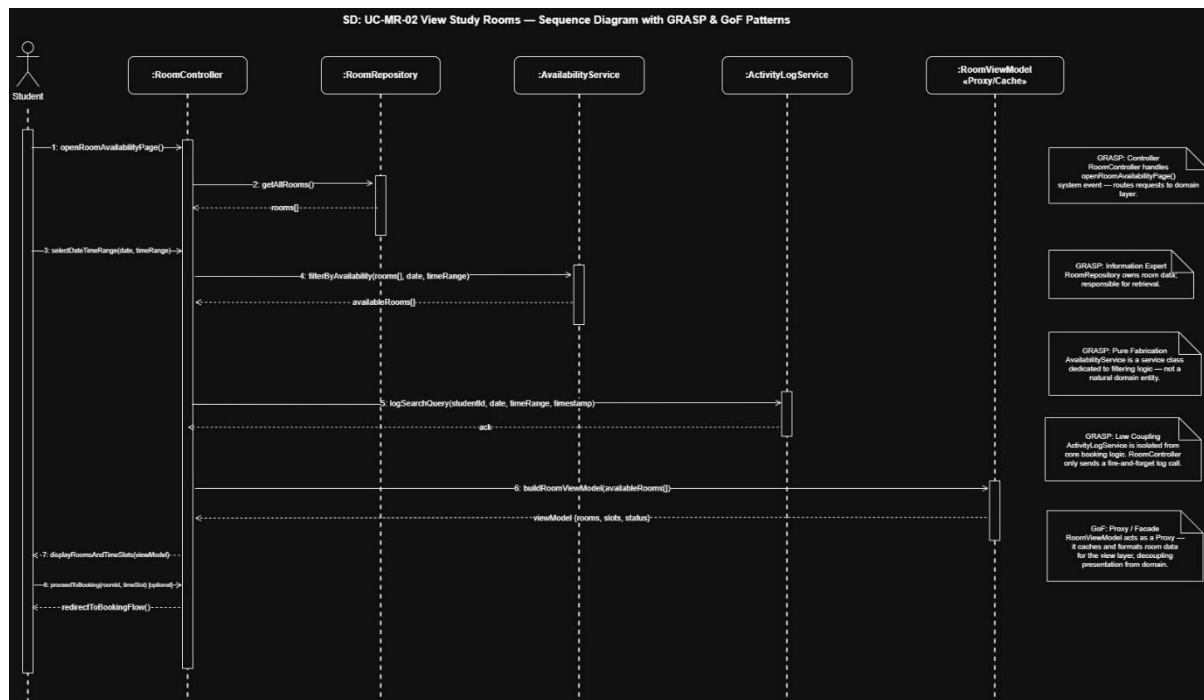
6. Sequence Diagrams

Sequence Diagrams show the internal object-level interactions for each use case, including GRASP and GoF pattern annotations on each diagram. Each SD demonstrates the call chain from the Web Handler layer through business controllers, services, repositories, and back, with design pattern justifications annotated directly on the diagram.

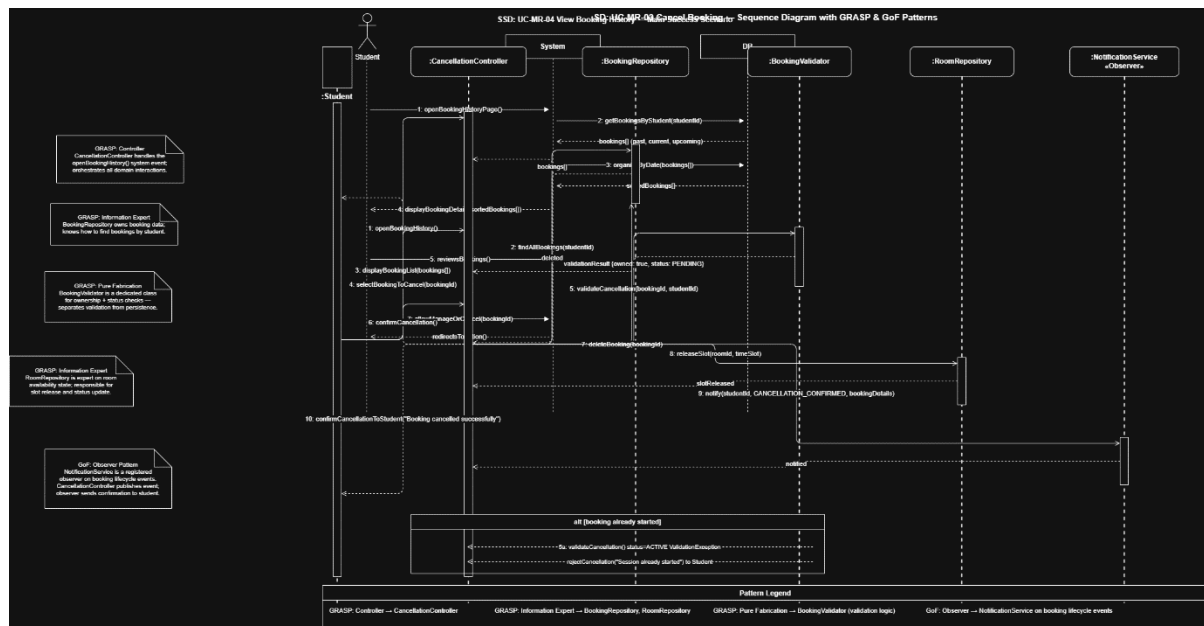
UC-MR-01: Request Study Room



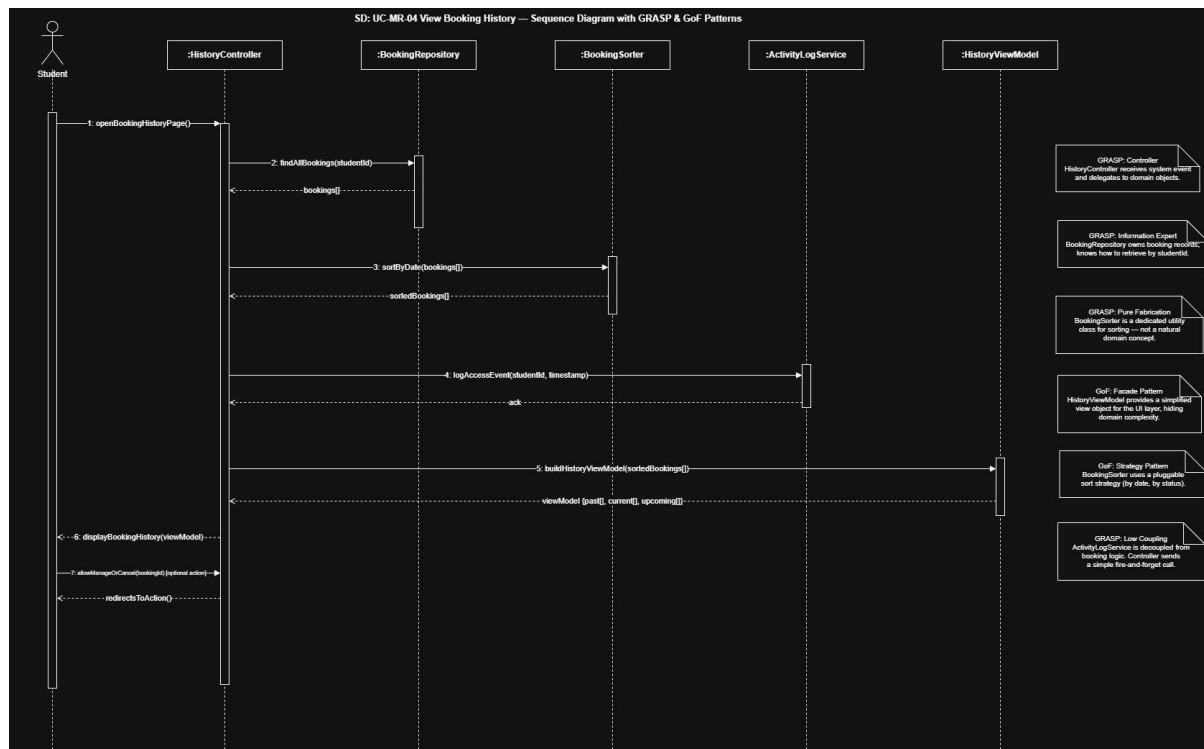
UC-MR-02: View Study Rooms



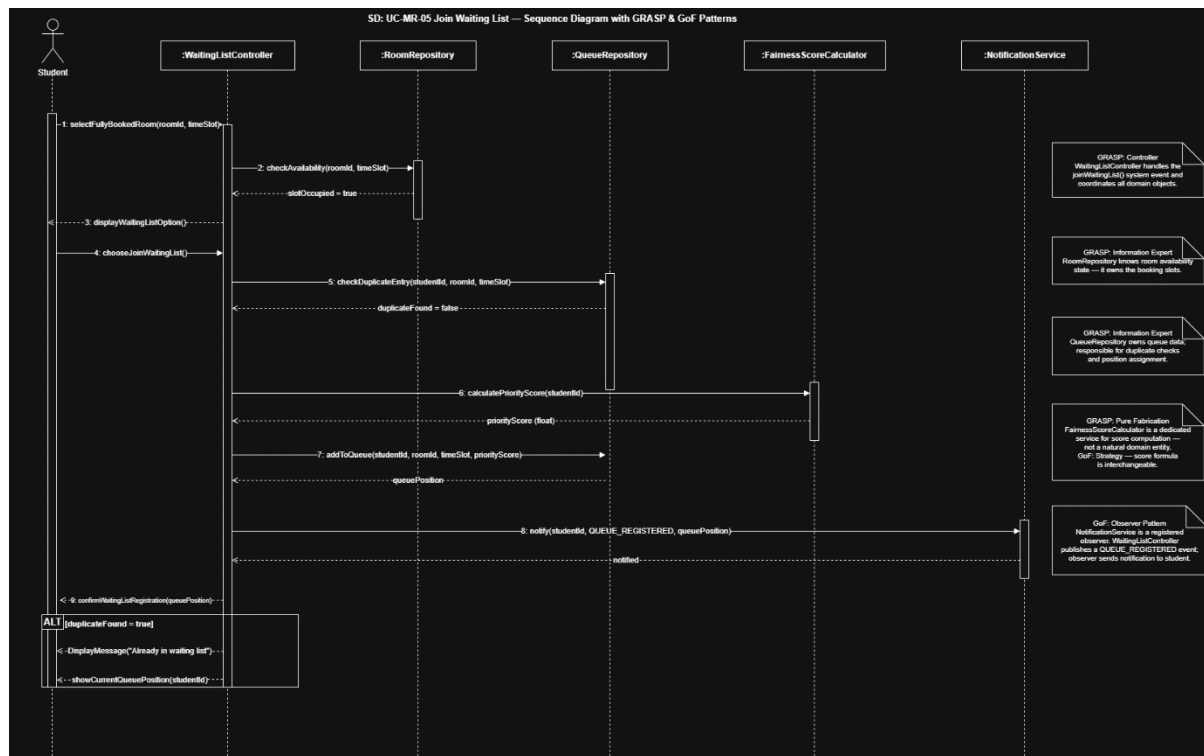
UC-MR-03: Cancel Booking



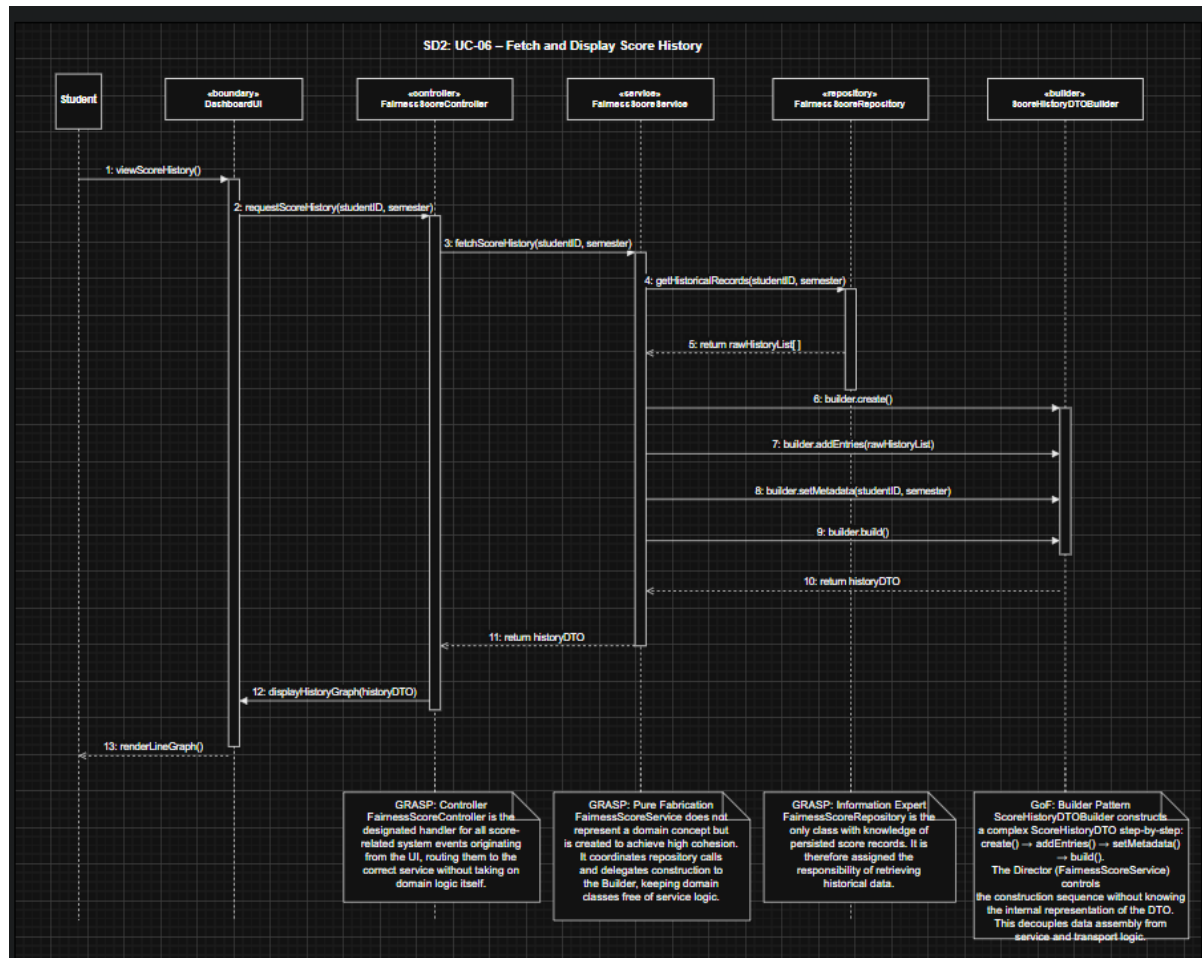
UC-MR-04: View Booking History



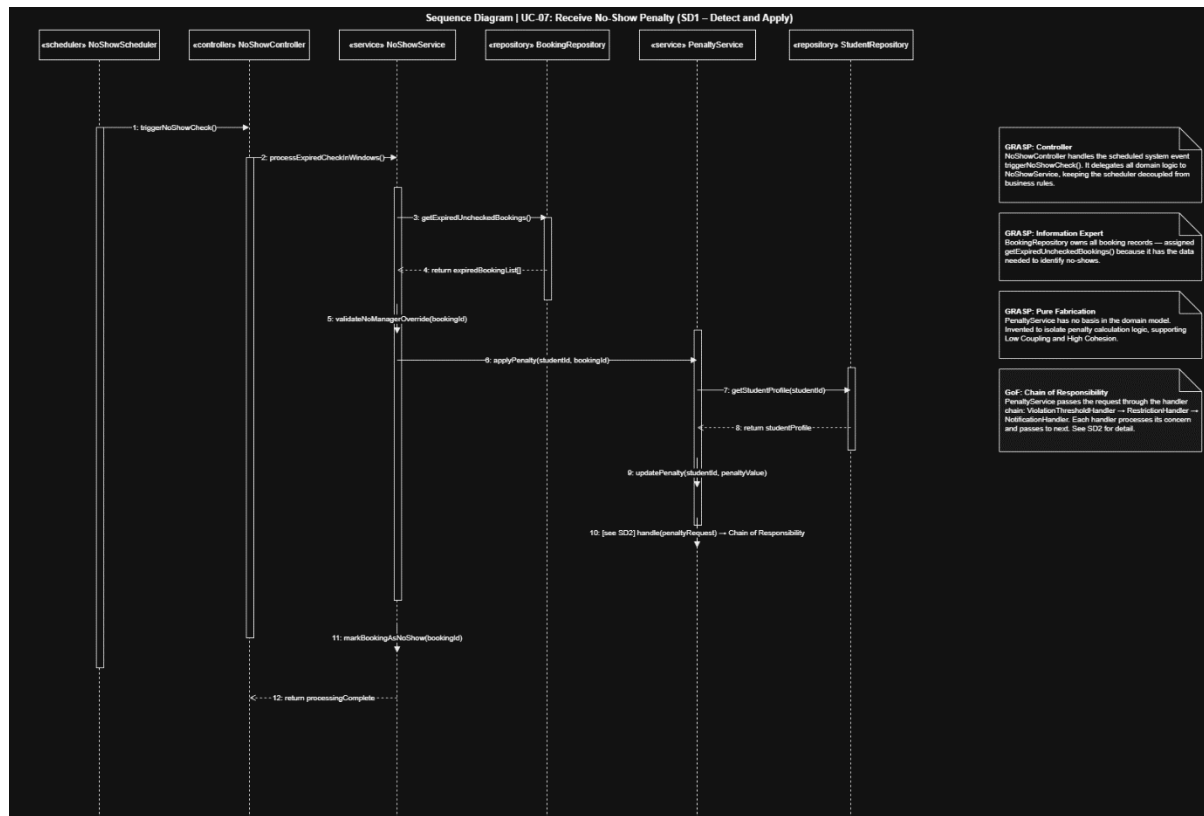
UC-MR-05: Join Waiting List



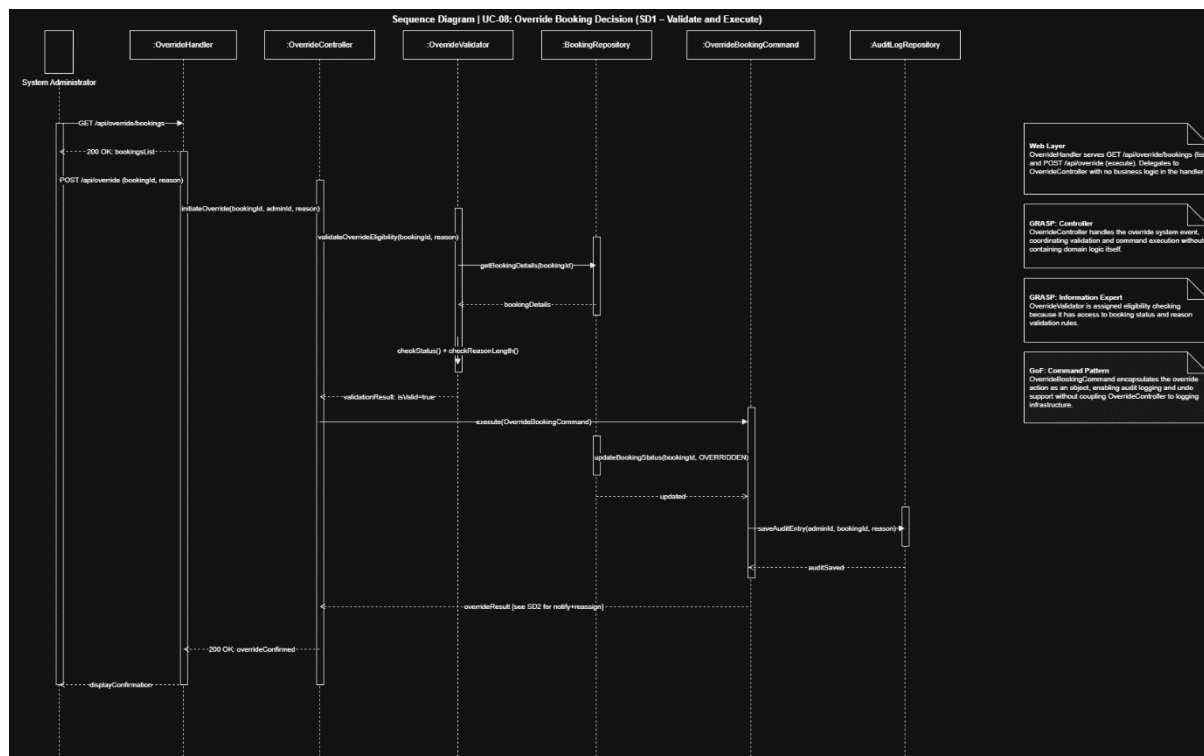
UC-06: View Fairness Score



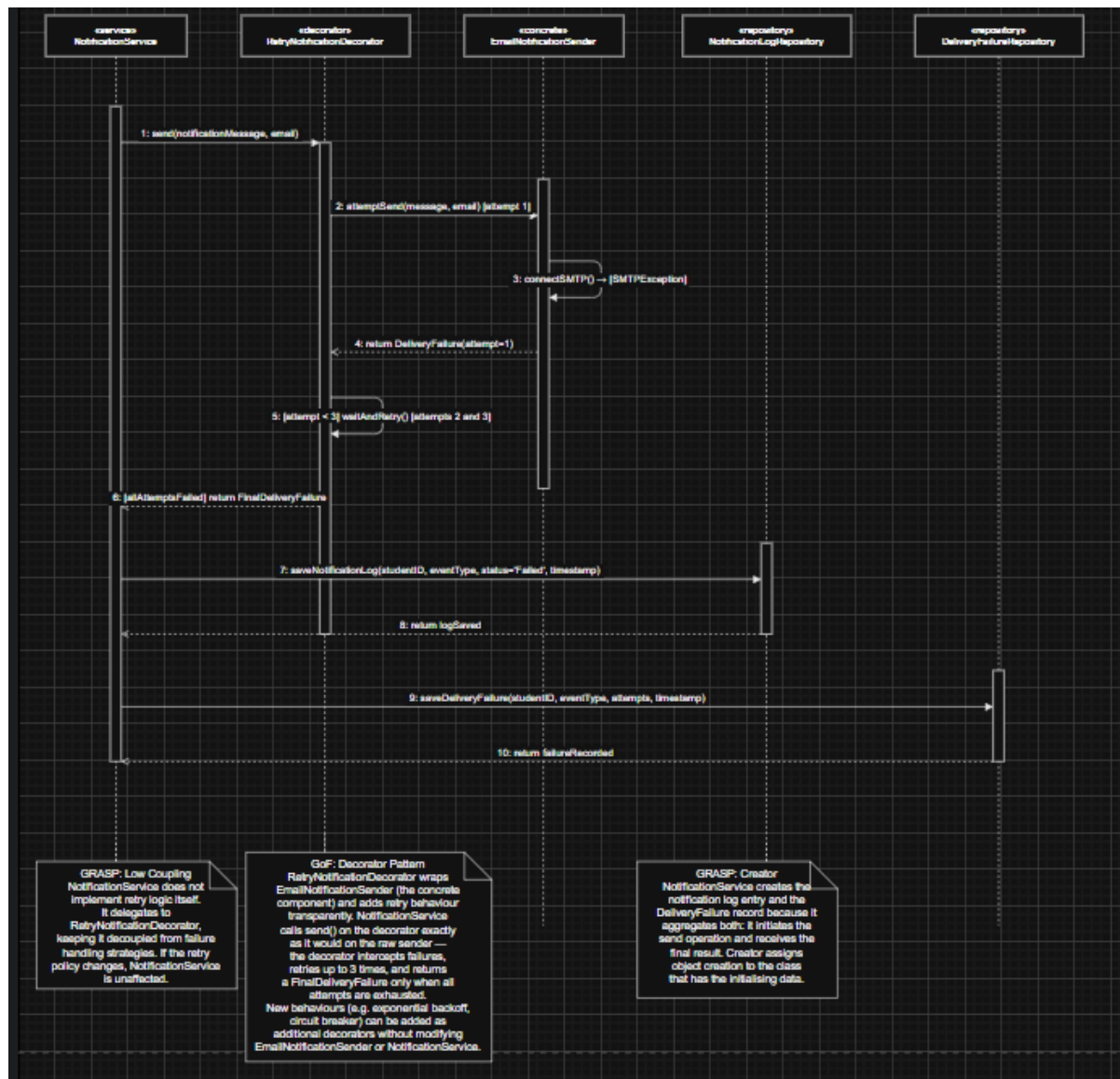
UC-07: Receive No-Show Penalty



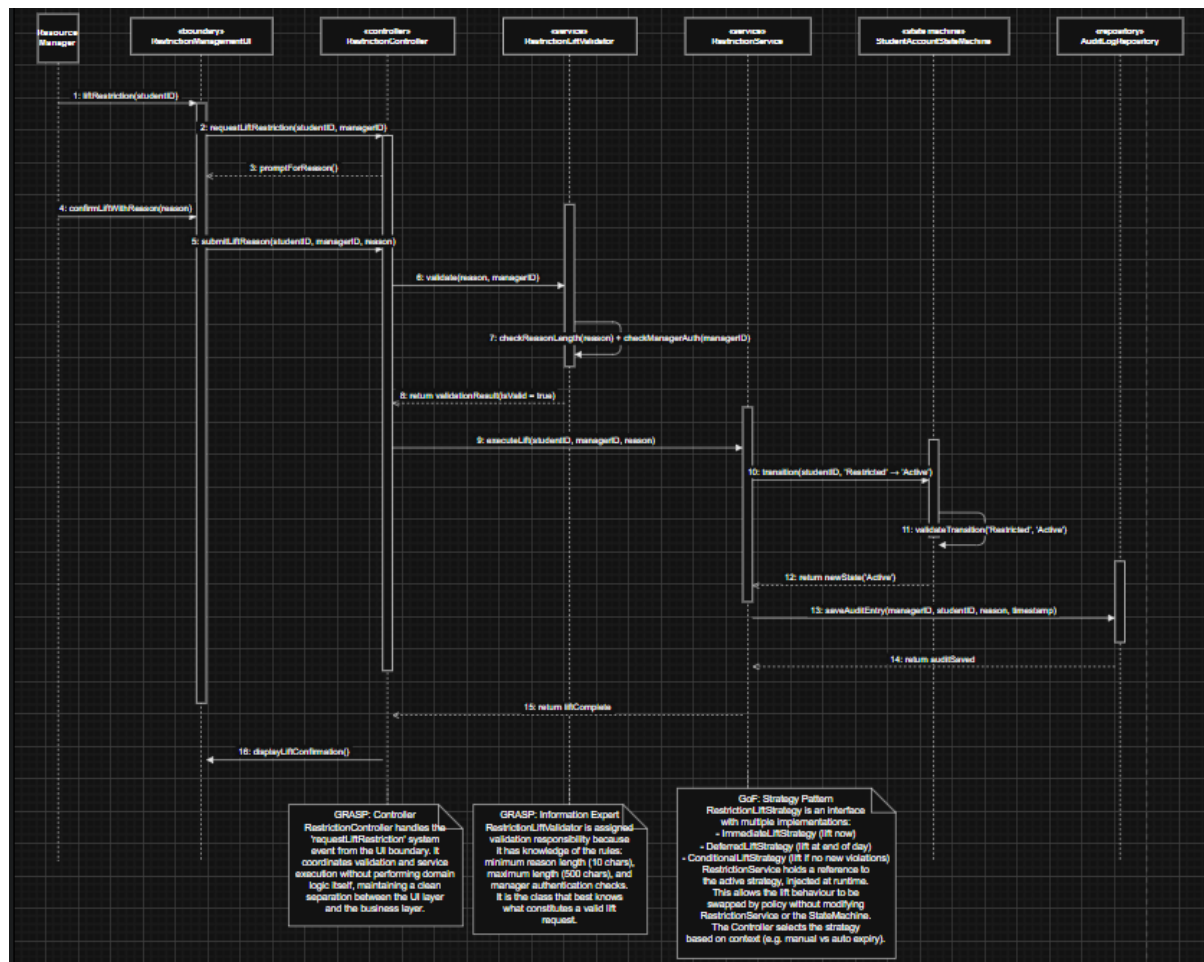
UC-08: Override Booking Decision



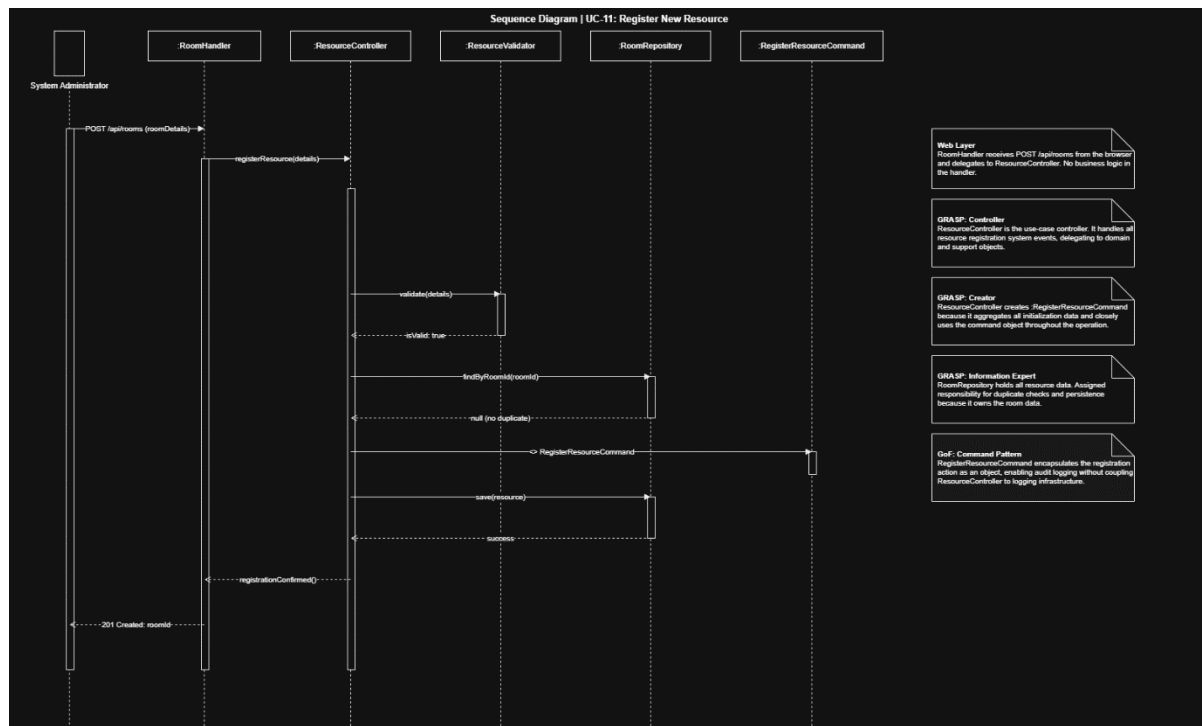
UC-09: Receive Booking Notification



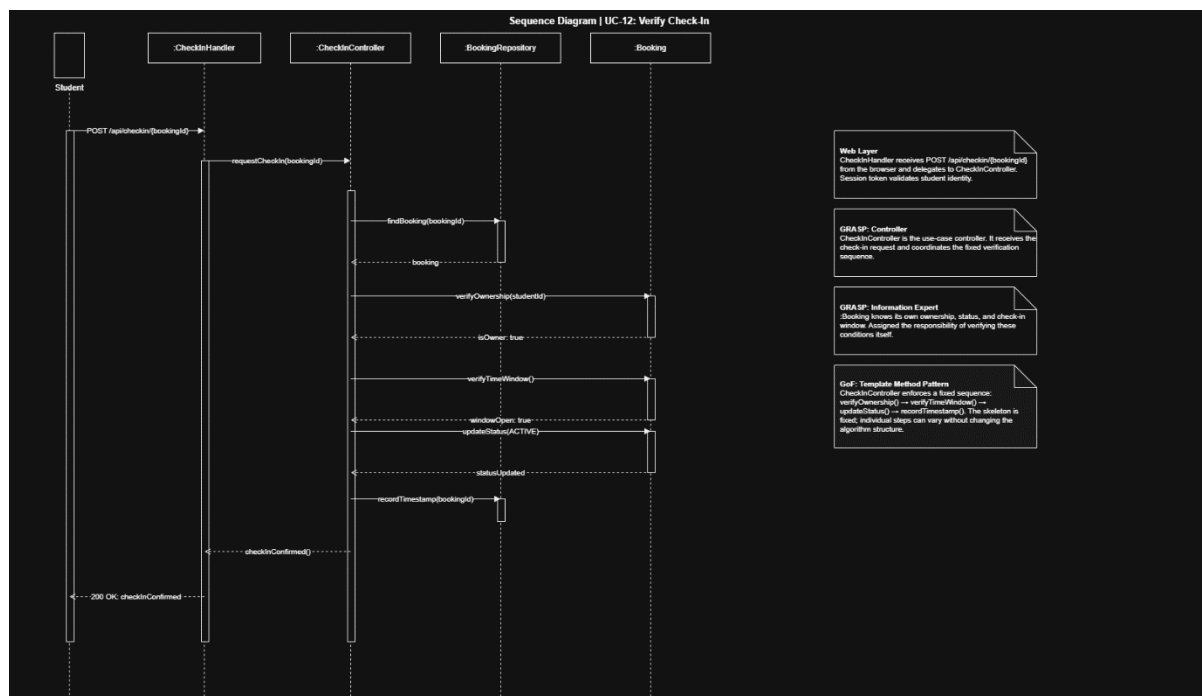
UC-10: Manage Account Restriction



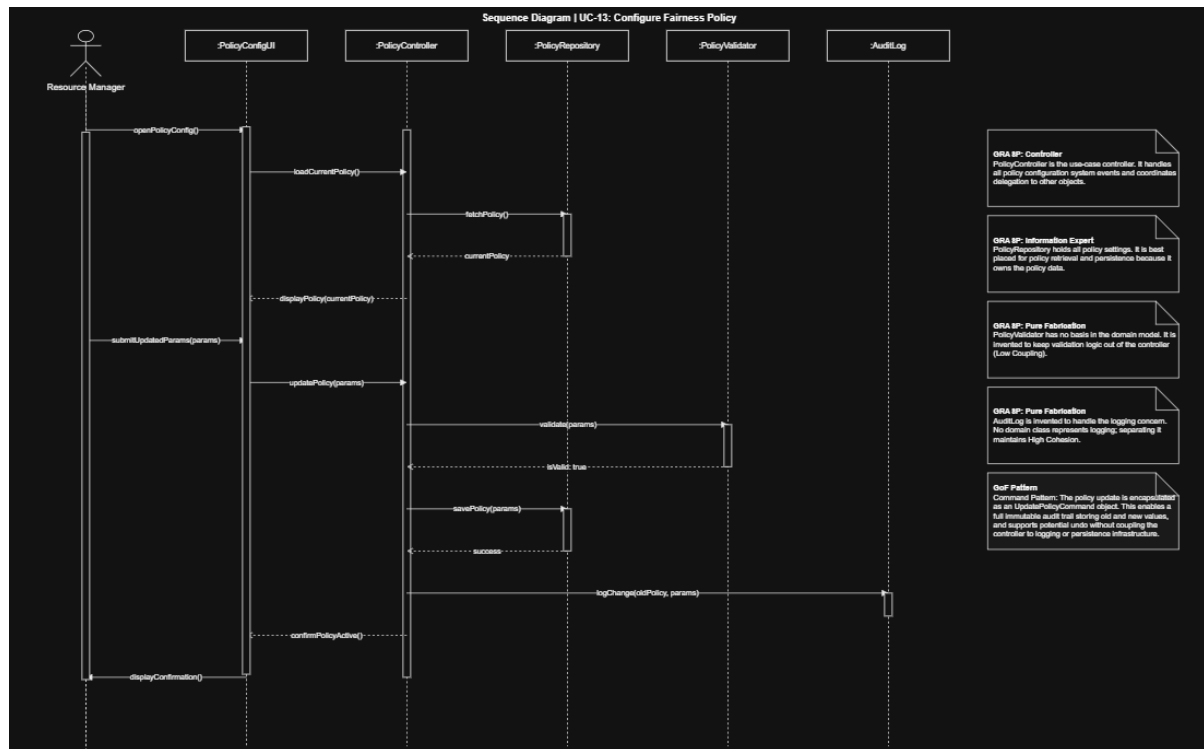
UC-11: Register New Resource



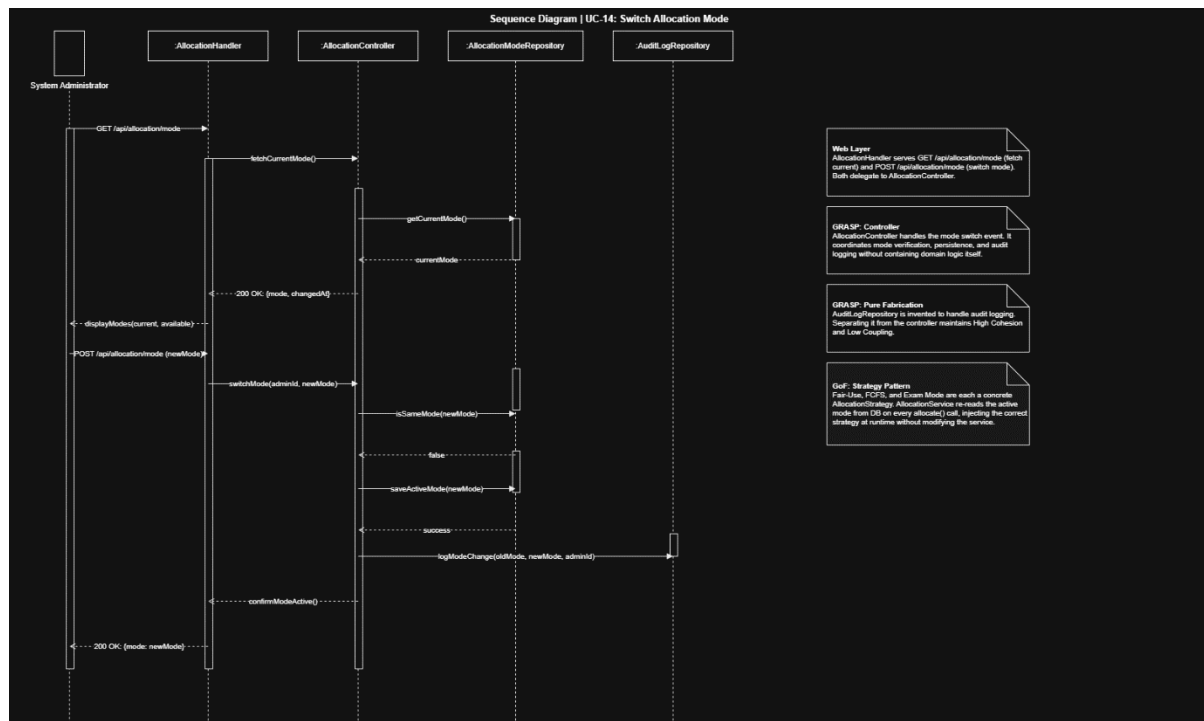
UC-12: Verify Check-In



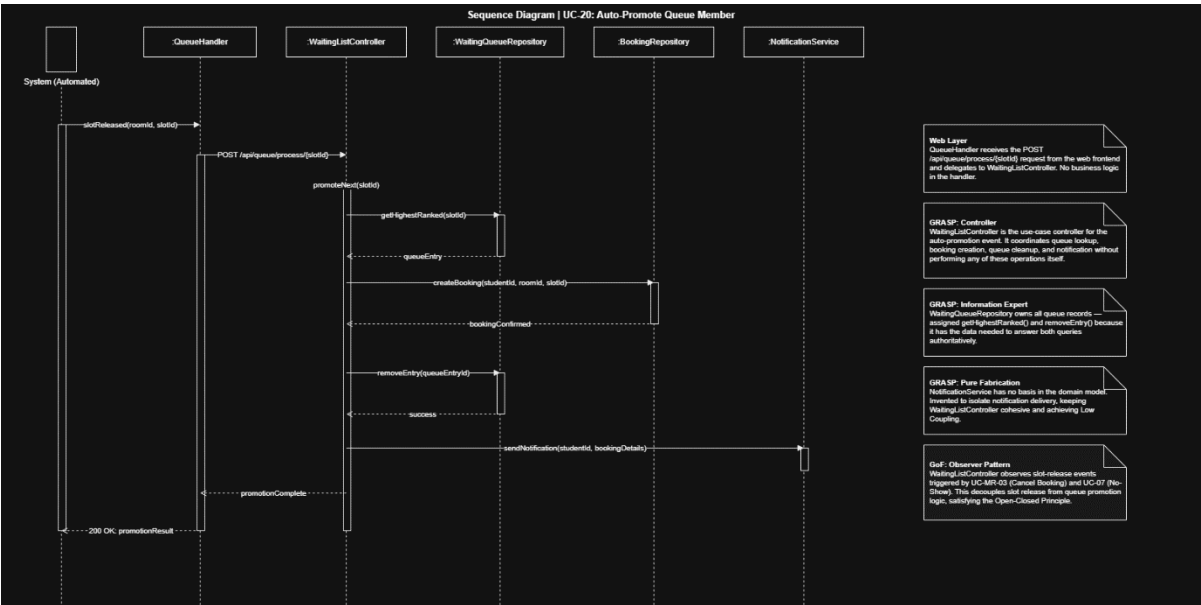
UC-13: Configure Fairness Policy



UC-14: Switch Allocation Mode

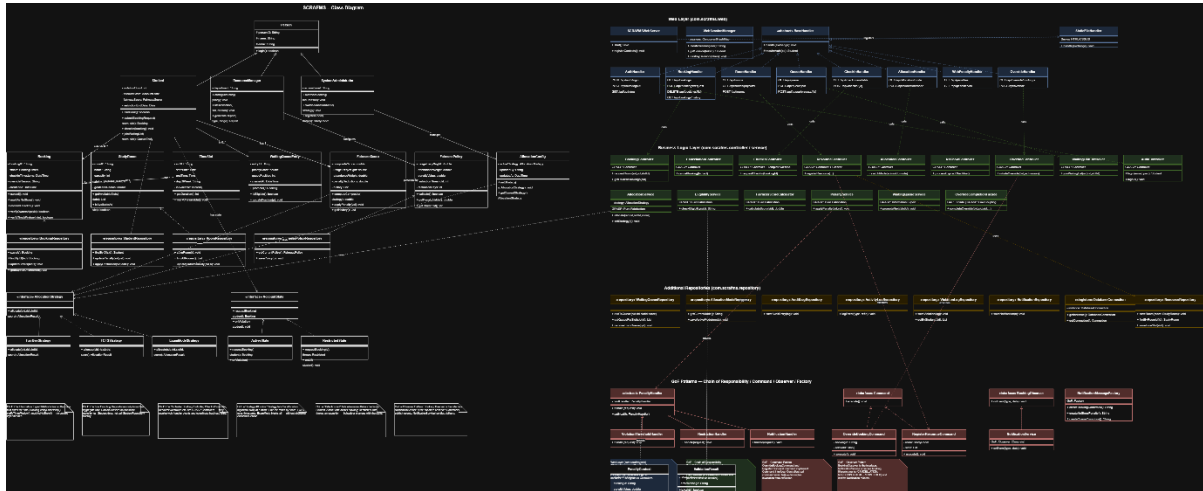


UC-15: Auto-Promote Queue Member



7. Class Diagram

The class diagram presents the complete static structure of SCRAFMS including all classes, attributes, methods, and relationships. It reflects the layered architecture and all applied GoF design patterns. The diagram is organised into five colour-coded sections: the original domain model and pattern classes (white), the new web layer classes added during implementation (light blue), business logic controllers and services (light green), additional repositories (light yellow), and GoF pattern classes (light red).



Key structural highlights:

- Person as the base class with Student, ResourceManager, and SystemAdministrator as concrete subclasses
- AllocationStrategy interface with FairUseStrategy, FCFSStrategy, and ExamModeStrategy implementations (Strategy pattern)
- AccountState interface with ActiveState and RestrictedState implementations (State pattern)
- PenaltyHandler abstract class with ViolationThresholdHandler, RestrictionHandler, and NotificationHandler (Chain of Responsibility)
- BookingObserver interface implemented by NotificationService (Observer pattern)
- Command interface implemented by OverrideBookingCommand and RegisterResourceCommand (Command pattern)
- Web handler classes (BaseHandler, AuthHandler, BookingHandler, etc.) added during implementation — not in original design
- All repository classes using DatabaseConnection singleton via JDBC

8. High-Level Architecture

SCRAFMS adopts a strict three-tier Layered Architecture. This architectural style was selected because it directly maps to the call chains established in the Sequence Diagrams, satisfies the rubric's requirement for three separate layers, enforces clean separation of concerns, and is appropriate for a single-machine web application of this scale.

Layer	Package	Responsibility
Presentation Layer	com.scrafms.web, src/main/resources/web/	Serves HTML/CSS/JS pages via Java HttpServer on port 8080. Handles HTTP requests and delegates to the business layer. No business logic in this layer.
Business Logic Layer	com.scrafms.controller, .service, .strategy, .handler, .command, .observer, .factory, .state	All use case logic, design patterns, allocation algorithms, penalty processing, and queue management.
Data Access Layer	com.scrafms.repository, .model, .util	JDBC repository classes, domain model POJOs, DatabaseConnection singleton.

Justification for Layered Architecture:

1. Direct mapping to Sequence Diagrams — every SD shows the call chain Web Handler → Business Controller → Service → Repository → DB. Layered Architecture formalises this as a first-class constraint.
2. Satisfies the three-layer requirement — Presentation (com.scrafms.web), Business Logic (com.scrafms.controller/service), and Data Access (com.scrafms.repository) are physically separate packages.
3. Testability — the business logic layer has no dependency on the HTTP server. Service and repository classes can be tested independently.
4. Appropriate scale — Event-Driven or Microservices architectures would introduce unnecessary complexity for a single-machine application.
5. Supports GRASP principles — GRASP Controller is realised at the boundary between Presentation and Business layers. GRASP Information Expert is realised within Repository and Model layers.

Note on web layer addition: The implementation introduces a Web API layer (com.scrafms.web) that was not present in the original Deliverable 2/3 design artefacts, which assumed a JavaFX UI. This layer replaces the JavaFX presentation layer with a Java HttpServer serving HTML/CSS/JS pages, in accordance with the project rubric which permits any front-end technology. All business logic classes, method signatures, and design patterns remain consistent with the Sequence Diagrams produced in Deliverable 3. The web handler classes act as a thin translation layer between HTTP requests and the existing business controller classes, preserving the integrity of the layered architecture.

The diagram illustrates the SCRAFMS deployment architecture, showing the flow of data and control between various components across different machines.

Developer/User Machine (Windows 10/11, 64-bit)

- Web Browser (Chrome / Firefox / Edge)**
 - login.html
 - student-dashboard.html
 - manager-dashboard.html
 - admin-dashboard.html
 - shared.js + style.css
- JVM Process (OpenJDK 21)**
 - SCRAFMS Application (Maven: com.scrafms.Main)**
 - java HttpServer (com.sun.net.httpserver) Port: 8080
 - Business Logic Layer (Pure Java, No Frameworks)**
 - Controllers
 - Services
 - Strategies
 - Handlers Chain
 - Commands
 - Observer
 - Data Access Layer (JDBC)**
 - Repositories
 - DatabaseConnection Singleton
 - Background Thread (Daemon, No-Show Detection)
 - periodic no-show check
 - repository calls (java)
 - method calls (java)
 - schema.sql (database schema)
 - db.properties (connection config)
 - need on startup

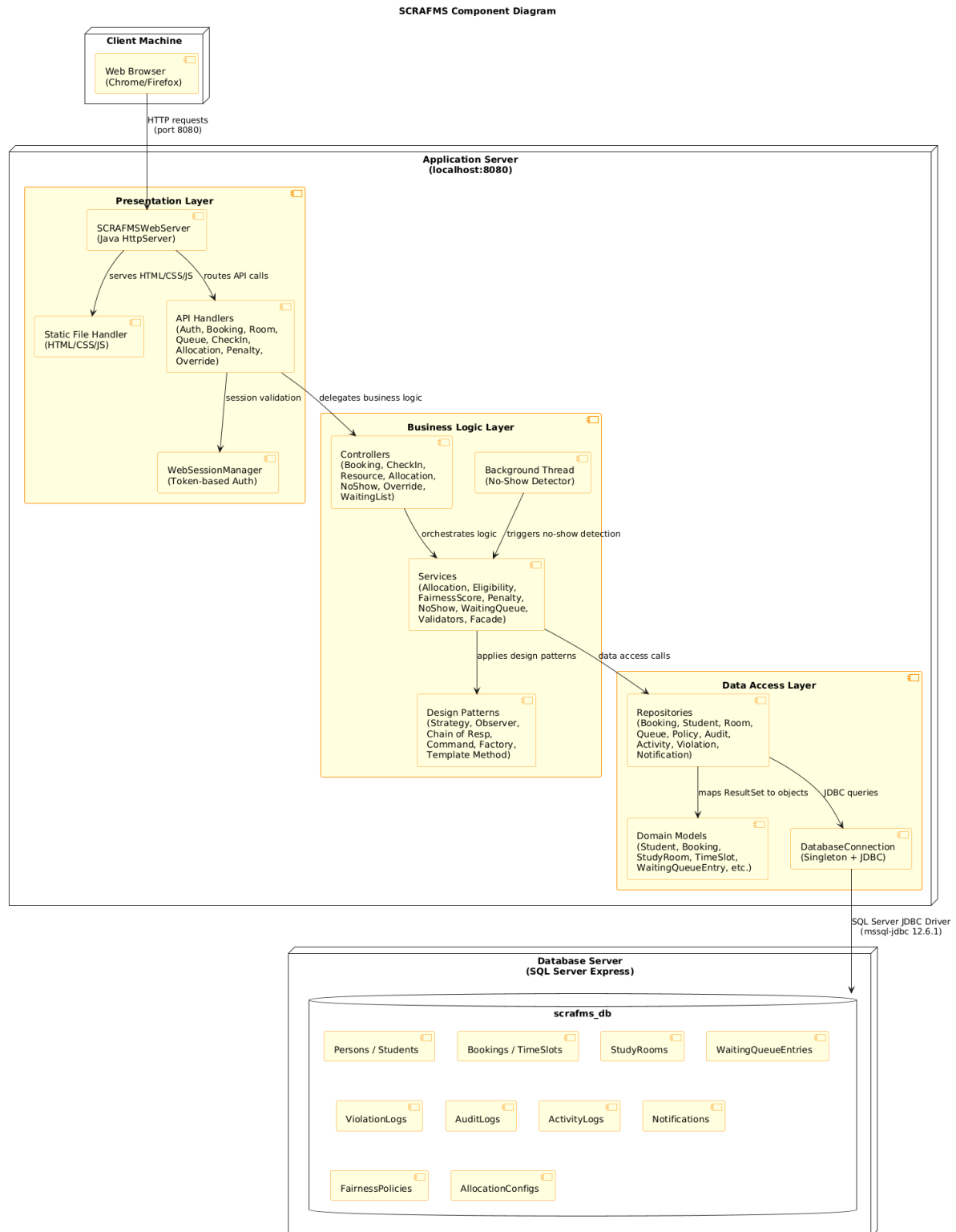
SQL Server Express (DESKTOP-MN1LGO7\SQLEXPRESS)

- scrafms_db
 - 15 Tables
 - Seed Data
 - Indexes + FK Constraints

Annotations:

- Single Page Application: Vanilla HTML/CSS/JS, No Framework, dependencies fetch() API for all requests, Cookie-based session, auth.
- Design Patterns Applied: Strategy (Allocation), Observer (Notifications), Chain of Responsibility (Penalty), Command (Override, Register), Facade (Override Completion), Template Method (Check-in), Singleton (DB Connection), Factory (Notifications).
- executed once on setup
- JDBC (mssql-jdbc 12.6.1 TCP/IP port 1433)

8.3 Component Diagram



9. Work Division Table

Student Name	Roll Number	Main Use Cases (Implemented)	Responsibilities
Mussa Raza	24I-3022	UC-MR-01 Request Study Room UC-MR-03 Cancel Booking UC-MR-05 Join Waiting List	Allocation engine, eligibility check, fairness score (UC-MR-01); booking validator, observer pattern, auto-promote trigger (UC-MR-03); queue management, duplicate prevention (UC-MR-05). Also implemented indirectly: UC-MR-02 (view rooms), UC-MR-04 (booking history).
Muhammad Zain	24I-3126	UC-07 No-Show Penalty UC-08 Override Booking UC-11 Register New Resource	Background thread, chain of responsibility, penalty calculation, restriction trigger (UC-07); command pattern, facade pattern, audit log (UC-08); resource validator, command pattern, time slot management (UC-11). Also implemented indirectly: UC-06 (fairness score), UC-10 (account restriction).
Ahmed Jamal	24I-3006	UC-12 Verify Check-In UC-14 Switch Allocation Mode UC-15 Auto-Promote Queue Member	Template method pattern, time window logic, check-in validation (UC-12); strategy injection, audit logging, runtime mode switching (UC-14); queue promotion logic, observer notification, integration with cancel and no-show flows (UC-15). Also implemented indirectly: UC-09 (notifications).

9.1 Design Patterns Summary

Pattern	Category	Classes	Use Cases
Strategy	GoF Behavioural	AllocationStrategy, FairUseStrategy, FCFSSStrategy, ExamModeStrategy	UC-MR-01, UC-14, UC-15
Observer	GoF Behavioural	BookingObserver, NotificationService	UC-MR-03, UC-MR-05, UC-07, UC-09, UC-15
Chain of Responsibility	GoF Behavioural	PenaltyHandler, ViolationThresholdHandler, RestrictionHandler, NotificationHandler	UC-07, UC-10
Command	GoF Behavioural	Command, OverrideBookingCommand, RegisterResourceCommand	UC-08, UC-11
Facade	GoF Structural	OverrideCompletionFacade	UC-08

Pattern	Category	Classes	Use Cases
Template Method	GoF Behavioural	CheckInController.requestCheckIn()	UC-12
Singleton	GoF Creational	DatabaseConnection	All UCs
Factory	GoF Creational	NotificationMessageFactory	UC-07, UC-08, UC-09, UC-15
State	GoF Behavioural	AccountState, ActiveState, RestrictedState	UC-10
Controller	GRASP	BookingController, CancellationController, CheckInController, ResourceController, AllocationController, NoShowController, OverrideController, WaitingListController	All UCs
Information Expert	GRASP	BookingRepository, StudentRepository, RoomRepository, WaitingQueueRepository, etc.	All UCs
Creator	GRASP	ResourceController creates Resource	UC-11
Pure Fabrication	GRASP	AllocationService, EligibilityService, FairnessScoreCalculator, PenaltyService, WaitingQueueService, ResourceValidator, BookingValidator, OverrideValidator, NotificationService, NotificationMessageFactory	All UCs
Low Coupling	GRASP	OverrideController → OverrideCompletionFacade, CancellationController → WaitingQueueService	UC-MR-03, UC-08
High Cohesion	GRASP	Each handler in the Chain of Responsibility has a single focused responsibility	UC-07
Protected Variation	GRASP	AllocationService wraps strategy behind interface; handler chain allows new steps without modifying existing	UC-MR-01, UC-07, UC-14